

3. Analízis modell

62 - banducole

Konzulens:

Szabó András

Csapattagok

Tullner Levente Botond	APJGDR	1234tl12@gmail.com
Dekkert Bence	S2V3EZ	deki.bence@gmail.com
Kiss Ákos Attila	FTS6X7	kissakos2005@gmail.com
Maruzsán Borisz	UJO4ZE	maruzsanborisz05@gmail.com
Stumpf Ádám György	NFTG9H	stumpfadam05@gmail.com

2026.03.08

3. Analízis modell kidolgozása

3.1 Objektum katalógus

3.1.1 Game

A játék objektum tárolja a várost, a játékosokat és az aktuális kör számot. Felelőssége a játék elindítása, valamint annak megállapítása, hogy véget ért-e a játék. Nem tartalmaz játék logikát.

3.1.2 City

A város tárolja a kereszteződések és az utakat. Felelőssége a végrehajtási lépések szabályozása.

3.1.3 PathFinder

Az útvonalkereső tárolja a várost. Felelőssége az útvonalkeresési logika, meghatározza a legrövidebb utat két csomópont között, visszaadja egy adott sáv szomszédait, az út másik végpontját, valamint a járható sávokat. Minden autó saját útvonalkeresővel rendelkezik.

3.1.4 BusDriver

A buszvezető tárolja a hozzárendelt buszt. Egyetlen felelőssége a kör során meghívott döntési metódus végrehajtása, amelyben meghatározza, hogy a busz a következő körben melyik sávon haladjon.

3.1.5 CleanerPlayer

A takarító tárolja a pénz egyenlegét és a rendelkezésére álló hókotrókat. Felelőssége a kör során hókotróit irányítani, ehhez meg kell adnia, melyik hókotró melyik sávra menjen. Tud új hókotró fejeket és hajtóanyagot vásárolni, új hókotrórt venni, valamint fogadni a letisztított sávokért járó pénzt.

3.1.6 Road

Az út tárolja a két végpontját és az úton lévő sávokat. Felelőssége a sávjainak kezelése és a hóesés továbbítása a sávokra.

3.1.7 Intersection

A kereszteződés tárolja a hozzá csatlakozó utakat. Felelőssége, hogy nyilvántartása a kapcsolódó utakat, és azokat visszaadja lekérdezéskor, lehetővé téve az útvonalkeresést a városban. Új utat is lehet hozzáadni.

3.1.8 Terminal

A végállomás a buszok útvonalának egyik végpontja. Felelőssége, hogy értesítést fogadjon, amikor egy busz megérkezik hozzá. Csomópontként illeszkedik az úthálózatba, visszaadja a csatlakozó utakat.

3.1.9 Home

A lakás az autók egyik célpontja az úthálózaton. Csomópontként illeszkedik a hálózatba, visszaadja a csatlakozó utakat. Saját logikával nem rendelkezik.

3.1.10 Workplace

A munkahely az autók másik célpontja az úthálózaton. Csomópontként illeszkedik a hálózathoz, visszaadja a csatlakozó utakat. Saját logikával nem rendelkezik.

3.1.11 Lane

A sáv tárolja az aktuális állapotát, a rajta tartózkodó járműveket, az esetlegesen rajta lévő hókotrót, a sózás visszaszámlálóját és az áthaladások számát. Felelőssége járművek és hókotró fogadásának ellenőrzése, hó hozzáadása, só alkalmazása, jégképződés kezelése, a sáv blokkolása ütközés esetén, valamint a hókotró általi takarítás elvégzése. A sávnak lekérdezhető az aktuális állapota.

3.1.12 ClearState

A sáv alapállapota, amelyben sem hó, sem jég nincs. Átjárható és nem csúszós. Hó hatására vékony hó állapotba vált. Só hatására védett lesz. Hókotró nem tud rajta takarítani.

3.1.13 ThinSnowState

A sáv állapota, amelybe sima sávra hulló hó hatására kerül. Tárolja a hóréteg vastagságát. Átjárható és nem csúszós. További hó hatására vastag hó állapotba vált. Ha elég jármű halad át rajta, jeges állapotba vált. Sóval olvasztható.

3.1.14 ThickSnowState

A sáv állapota, amelybe a hóréteg tovább vastagodásával kerül. Tárolja a hóréteg vastagságát. Járművek számára elakadást okoz, de hókotró számára átjárható. Sóval és bizonyos fejet használva hókotróval eltávolítható.

3.1.15 IcyState

A sáv állapota, amelybe vékony havas sávon sok jármű áthaladása után kerül. Járható, de csúszós, a járművek megcsúszhatnak rajta. Hó ráhullása vastagítja a jégréteget, de nem változtat az állapotban. Sóval vagy sárkányfejjel olvasztható, jégtörővel feltört jég állapotba hozható.

3.1.16 BrokenIceState

A sáv állapota, amelybe jégtörő fejjel kerül. Tárolja a jégréteg vastagságát. Járható, de csúszós, akárcsak a jeges állapot. Különbség, hogy söprő és hánycsúszó fejjel eltávolítható, ellentétben a jéggel.

3.1.17 Car

Az autó tárolja a saját lakását, munkahelyét, és az aktuális sávját. Felelőssége, hogy körönként önállóan meghatározza a következő sávját az elérhető utak alapján, és szükség esetén sávot váltson. Az autót a játékosok nem irányítják.

Ha jeges sávon halad, csúszni kezd és ütközhet más járművekkel. Ha vastag hóba kerül, elakad.

3.1.18 Bus

A busz tárolja a két végállomását, az aktuális sávját és a megtett fordulókat számát. Felelőssége nyilvántartani, hogy megérkezett-e egy végállomásra, lekérdezhető a megtett forduló száma, és hogy teljesítette-e az elvárt útvonalat.

A busz mozgását a buszvezető határozza meg. Ütközhet más járművekkel jeges sávon.

3.1.19 SnowPlow

A hókotró tárolja az aktuális sávját, a felszerelt hókotró fejét és a tulajdonos takarítót. Felelőssége, hogy a takarító utasítására egy megadott sávra mozduljon, elvégezze a takarítást a felszerelt fejjel, esetleg fejet cseréljen.

3.1.20 SweepHead

Olyan hókotrófej, amely a havat és a feltört jeget a jobbra szomszédos sávba tolja (vagy az út mellé, ha jobb szélső sávban van). A szomszéd sáv hóvastagsága megnő a letolt hótól. A jegen nem működik. Nem igényel üzemanyagot.

3.1.21 ThrowHead

Olyan hókotrófej, amely a havat és a feltört jeget minden esetben az út mellé szórja, nem a szomszéd sávba. A jegen nem működik. Nem igényel üzemanyagot, ezért mindig működőképes.

3.1.22 IceBreakerHead

Olyan hókotrófej, amely a jégpáncélt feltört jég állapotba hozza. Havon és feltört jégen nem végez takarítást. Nem igényel üzemanyagot, ezért mindig működőképes.

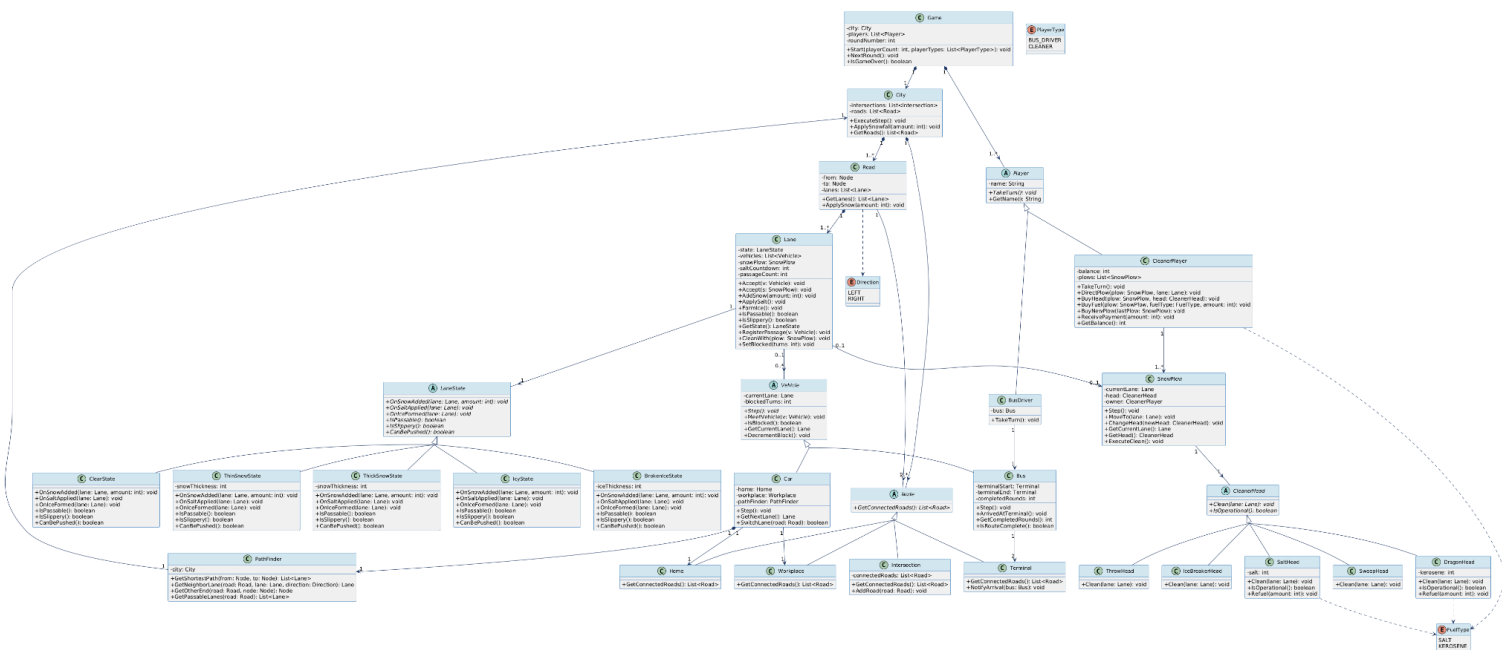
3.1.23 SaltHead

Olyan hókotrófej, amely sót szór a sávra. Tárolja a maradék sókészletet. A só hatása a sávon a sáv felelőssége. Ha a só készlet elfogy, a fej nem működőképes. Újratölthető sóval.

3.1.24 DragonHead

Olyan hókotrófej, amely gázturbinával azonnal megtisztítja a sávot. Tárolja a maradék biokerozin készletet. Ha a biokerozin elfogy, a fej nem működőképes. Újratölthető biokerozinnal.

3.2 Statikus struktúra diagramok



3.3 Osztályok leírása

3.3.1 BrokenIceState

- Felelősség**

A feltört jég állapotot reprezentálja. Akkor kerül egy sáv ebbe az állapotba, ha jégtörő fejjel dolgoztak rajta. Átjárható, de csúszós. Söprő és hányó fejjel eltávolítható.

- Ősosztyúk**

LaneState

- Interfészek**

- Asszociációk**

- Attribútumok**

- iceThickness: int:** a jégréteg vastagsága

- Metódusok**

- OnSnowAdded(lane: Lane, amount: int): void:** hó hozzáadásakor vastagítja a jégréteget, de az állapot nem változik
- OnSaltApplied(lane: Lane): void:** só hatására a jég olvad, állapota ClearState-be megy
- OnIceFormed(lane: Lane): void:** jégképződés itt nem értelmezett, nincs hatás
- IsPassable(): boolean:** mindig igaz, a sáv járható

- **IsSlippery(): boolean:** mindig igaz, a sáv csúszós
- **CanBePushed(): boolean:** igaz, megfelelő fejfel eltávolítható

3.3.2 Bus

- **Felelősség**

Egy buszt modellez, amely két végállomás között ingázik. A buszvezető irányítja körönként. Nyilvántartja a megtett fordulókat számát, és jelzi az érkezést a végállomásnak.

- **Ősosztályok**

Vehicle

- **Interfészek**

- **Asszociációk**

- **currentLane: Lane:** az aktuális sáv (örökölt)

- **Attribútumok**

- **terminalStart: Terminal:** az útvonal kezdő végállomása
- **terminalEnd: Terminal:** az útvonal záró végállomása
- **completedRounds: int:** eddig teljesített teljes fordulókat száma
- **currentLane: Lane:** az aktuális sáv (örökölt)
- **blockedTurns: int:** hány körre van blokkolva a jármű (örökölt)

- **Metódusok**

- **Step(): void:** a buszvezető döntése alapján a busz a következő sávra lép
- **ArrivedAtTerminal(): void:** értesíti a végállomást az érkezésről, növeli a fordulószámlálót
- **GetCompletedRounds(): int:** visszaadja a teljesített fordulókat számát
- **IsRouteComplete(): boolean:** igaz, ha az elvárt számú fordulókat teljesítette
- **MeetVehicle(v: Vehicle): void:** ütközés kezelése jeges sávon (örökölt)
- **IsBlocked(): boolean:** igaz, ha a busz ütközés miatt elakadt (örökölt)
- **GetCurrentLane(): Lane:** visszaadja az aktuális sávot (örökölt)
- **DecrementBlock(): void:** csökkenti az elakadás visszaszámlálóját (örökölt)

3.3.3 BusDriver

- **Felelősség**

A buszvezető játékost reprezentálja. Körönként eldönti, hogy a hozzárendelt busz melyik sávra haladjon a következő lépésben.

- **Ősosztályok**

Player

- **Interfészek**

- **Asszociációk**
 - **bus: Bus:** az irányított busz
- **Attribútumok**
 - **bus: Bus:** a buszvezető által irányított busz
 - **name: String:** a játékos neve (örökölt)
- **Metódusok**
 - **TakeTurn(): void:** a buszvezető meghozza a kör döntését: megadja a busznak a következő sávot (örökölt)
 - **GetName(): String:** visszaadja a játékos nevét (örökölt)

3.3.4 Car

- **Felelősség**

Egy személyautót modellez, amely lakása és munkahelye között ingázik. Önállóan navigál a PathFinder segítségével, a játékosok nem irányítják. Jeges sávon megcsúszhat és ütközhet, vastag hóban elakad.

- **Ősosztályok**

Vehicle

- **Interfészek**
-

- **Asszociációk**
 - **home: Home:** a lakóhely csomópontja, az útvonal egyik végpontja
 - **workplace: Workplace:** a munkahely csomópontja, az útvonal másik végpontja
 - **pathFinder: PathFinder:** a saját útvonalkereső példány, amelyet navigációhoz használ
 - **currentLane: Lane:** az aktuális sáv (örökölt)
- **Attribútumok**
 - **home: Home:** a lakóhely csomópont
 - **workplace: Workplace:** a munkahely csomópont
 - **pathFinder: PathFinder:** az autó saját útvonalkeresője
 - **currentLane: Lane:** az aktuális sáv (örökölt)
 - **blockedTurns: int:** hány körre van blokkolva a jármű (örökölt)
- **Metódusok**
 - **Step(): void:** meghatározza és végrehajtja a következő lépést (örökölt)
 - **GetNextLane(): Lane:** a PathFinder segítségével kiszámítja a következő sávot
 - **SwitchLane(road: Road): boolean:** sávváltást hajt végre a megadott úton. Visszaadja, hogy sikeres volt-e
 - **MeetVehicle(v: Vehicle): void:** ütközés kezelése jeges sávon (örökölt)
 - **IsBlocked(): boolean:** igaz, ha elakadt (örökölt)
 - **GetCurrentLane(): Lane:** visszaadja az aktuális sávot (örökölt)
 - **DecrementBlock(): void:** csökkenti a blokkolás visszaszámlálóját (örökölt)

3.3.5 City

- **Felelősség**

A város a közlekedési hálózat topológiai adattárolója. Tárolja az utakat és csomópontokat. Szabályozza a lépéseket. Alkalmazza a hóesést.

- **Ősosztályok**

- **Interfészek**

- **Asszociációk**

- **roads: List<Road>**: az összes út a városban
- **intersections: List<Intersection>**: az összes kereszteződés a városban

- **Attribútumok**

- **roads: List<Road>**: az úthálózat útjainak listája
- **intersections: List<Intersection>**: a kereszteződések listája

- **Metódusok**

- **ExecuteStep(): void**: végigiterál az utakon és sávokon, meghívja minden jármű és hókotró Step() metódusát
- **ApplySnowfall(amount: int): void**: az összes útra alkalmazza a hóesést a megadott mennyiséggel
- **GetRoads(): List<Road>**: visszaadja az összes utat

3.3.6 ClearState

- **Felelősség**

A sáv alap állapotát reprezentálja, sem hó, sem jég nincs rajta. Átjárható és nem csúszós. Hó hatására vékony hó állapotba vált. Hókotró nem tud rajta takarítani.

- **Ősosztályok**

LaneState

- **Interfészek**

- **Asszociációk**

- **Attribútumok**

- **Metódusok**

- **OnSnowAdded(lane: Lane, amount: int): void**: hó hatására a sáv ThinSnowState-be vált

- **OnSaltApplied(lane: Lane): void:** só hatására a sáv védett lesz a jégképződéssel szemben
- **OnIceFormed(lane: Lane): void:** jégképződés ebben az állapotban nem értelmezett
- **IsPassable(): boolean:** mindig igaz
- **IsSlippery(): boolean:** mindig hamis
- **CanBePushed(): boolean:** hamis, nincs mit eltávolítani

3.3.7 CleanerHead

- **Felelősség**

Absztrakt osztály, amely a hókotró fejet reprezentálja. Minden fej képes egy sávon takarítást végezni, és lekérdezhető, hogy működőképes-e.

- **Ősosztályok**

- **Interfészek**

- **Asszociációk**

- **Attribútumok**

- **Metódusok**

- **Clean(lane: Lane): void:** elvégzi a takarítást az adott sávon, az alosztály határozza meg a pontos viselkedést
- **IsOperational(): boolean:** igaz, ha a fej üzemképes (van elegendő üzemanyag)

3.3.8 CleanerPlayer

- **Felelősség**

A takarító járműt kezelő játékost reprezentálja. Körönként irányítja a hókotróit, ezzel megadja, hogy melyik hókotró melyik sávra menjen. Vásárolhat új hókotró fejet, üzemanyagot és új hókotró. A letisztított sávokért fizetést kap.

- **Ősosztályok**

Player

- **Interfészek**

- **Asszociációk**

- **plows: List<SnowPlow>:** az irányított hókotrók listája

- **Attribútumok**

- **balance: int:** a pénzegeyenleg, amelyből vásárolhat felszerelést
- **plows: List<SnowPlow>:** a takarító rendelkezésére álló hókotrók

- **name: String:** a játékos neve (örökölt)
- **Metódusok**
 - **TakeTurn(): void:** a takarító meghozza a kör döntéseit (örökölt)
 - **DirectPlow(plow: SnowPlow, lane: Lane): void:** megadja, hogy az adott hókotró melyik sávra menjen
 - **BuyHead(plow: SnowPlow, head: CleanerHead): void:** új kotrófejet vásárol a megadott hókotróhoz
 - **BuyFuel(plow: SnowPlow, fuelType: FuelType, amount: int): void:** üzemanyagot tölt a megadott hókotróhoz
 - **BuyNewPlow(lastPlow: SnowPlow): void:** új hókotrót vásárol, hozzáadja a listához
 - **ReceivePayment(amount: int): void:** a letisztított sávokért járó fizetést fogadja, növeli az egyenleget
 - **GetBalance(): int:** visszaadja az aktuális pénz egyenleget
 - **GetName(): String:** visszaadja a játékos nevét (örökölt)

3.3.9 DragonHead

- **Felelősség**
Gázturbinás hókotró fej, amely azonnal megtisztítja a sávot. Mindent meg tud tisztítani. Biokerozin készlete véges, a takarító töltheti fel.

- **Ősosztályok**

CleanerHead

- **Interfészek**
-

- **Asszociációk**
-

- **Attribútumok**

- **kerosene: int:** a maradék biokerozin készlet mennyisége

- **Metódusok**

- **Clean(lane: Lane): void:** megtisztítja a sávot bármilyen hó vagy jég állapotból, ha van elég kerozin
- **IsOperational(): boolean:** igaz, ha a kerozin készlet nem üres
- **Refuel(amount: int): void:** feltölti a biokerozin készletet a megadott mennyiséggel

3.3.10 Game

- **Felelősség**

Tárolja a várost, a játékosok listáját és az aktuális kör számot. Felelőssége a játék elindítása, a körök sorrendben történő végrehajtása, és a játék végének megállapítása.

- **Ősosztályok**
-

- **Interfészek**

- **Asszociációk**

- **city: City:** a szimulált város
- **players: List<Player>:** a játékosok listája

- **Attribútumok**

- **city: City:** a játék városának modellje
- **players: List<Player>:** a résztvevő játékosok
- **roundNumber: int:** az aktuális kör száma

- **Metódusok**

- **Start(playerCount: int, playerTypes: List<PlayerType>): void:** inicializálja a játékot
- **NextRound(): void:** végrehajtja a következő kört: meghívja minden játékos TakeTurn()-jét
- **IsGameOver(): boolean:** megállapítja, hogy a játéknak vége van-e

3.3.11 Home

- **Felelősség**

Az autók lakóhelyi célállomása az úthálózaton. Csomópontként illeszkedik a hálózatba. Saját logikával nem rendelkezik, azonosítónak szolgál az autók navigációjában.

- **Ősosztályok**

Node

- **Interfészek**

- **Asszociációk**

- **connectedRoads:** a csomóponthoz csatlakozó utak (örökölt)

- **Attribútumok**

- **connectedRoads: List<Road>:** a csomóponthoz csatlakozó utak (örökölt)

- **Metódusok**

- **GetConnectedRoads(): List<Road>:** visszaadja a csomóponthoz csatlakozó utakat (örökölt)

3.3.12 IceBreakerHead

- **Felelősség**

Jégtörő hókotró fej, amely a jégpáncélt feltört jég állapotba hozza. Havon és feltört jégen nem végez takarítást. Nem igényel üzemanyagot.

- **Ősosztályok**

CleanerHead

- **Interfészek**

- **Asszociációk**

- **Attribútumok**

- **Metódusok**

- **Clean(lane: Lane): void:** IcyState esetén BrokenIceState-be hozza a sávot, más állapoton nincs hatása
- **IsOperational(): boolean:** mindig igaz, nem igényel üzemanyagot

3.3.13 Intersection

- **Felelősség**

Egy kereszteződést modellez, amelyhez több út csatlakozik. Nyilvántartja a kapcsolódó utakat, és lehetővé teszi új utak hozzáadását.

- **Ősosztályok**

Node

- **Interfészek**

- **Asszociációk**

- **connectedRoads: List<Road>:** a kereszteződéshez csatlakozó utak listája

- **Attribútumok**

- **connectedRoads: List<Road>:** a csatlakozó utak listája

- **Metódusok**

- **GetConnectedRoads(): List<Road>:** visszaadja a kereszteződéshez csatlakozó összes utat (örökölt)
- **AddRoad(road: Road): void:** új utat ad hozzá a kereszteződéshez

3.3.14 IcyState

- **Felelősség**

A jeges sáv állapotot reprezentálja. Vékony havas sávon sok jármű áthaladása után alakul ki. A sáv járható, de csúszós, a járművek megcsúszhatnak. Sóval vagy sárkányfejjel olvasztható, jégtörővel feltört jég állapotba hozható.

- **Ősosztályok**

LaneState

- **Interfészek**

- **Asszociációk**

- **Attribútumok**

- **Metódusok**

- **OnSnowAdded(lane: Lane, amount: int): void:** hó ráhullása vastagítja a jégréteget, de az állapot nem változik
- **OnSaltApplied(lane: Lane): void:** só hatására az állapot ClearState-be vált
- **OnIceFormed(lane: Lane): void:** jégképződés ebben az állapotban nem értelmezett
- **IsPassable(): boolean:** igaz, a sáv járható
- **IsSlippery(): boolean:** igaz, a sáv csúszós
- **CanBePushed(): boolean:** hamis, jeget nem lehet söprővel/hányóval eltávolítani

3.3.15 Lane

- **Felelősség**

Egy forgalmi sávot modellez egy úton belül. Kezeli az aktuális állapotot, nyilvántartja a rajta tartózkodó járműveket és az esetleges hókotrót. Felelős a hó és jég állapotváltozásaiért, az áthaladások számlálásáért és a blokkolás kezeléséért.

- **Ősosztályok**

- **Interfészek**

- **Asszociációk**

- **state: LaneState:** az aktuális állapot objektum
- **vehicles: List<Vehicle>:** a sávon tartózkodó járművek
- **snowPlow: SnowPlow:** az esetlegesen a sávon lévő hókotró

- **Attribútumok**

- **state: LaneState:** az aktuális sáv állapot
- **vehicles: List<Vehicle>:** a sávon tartózkodó járművek listája
- **snowPlow: SnowPlow:** a sávon lévő hókotró
- **saltCountdown: int:** hány körig véd a só a jégképződéstől
- **passageCount: int:** az áthaladások száma, jégképződéshez

- **Metódusok**

- **Accept(v: Vehicle): void:** ellenőrzi és engedélyezi a jármű belépését a sávra
- **Accept(s: SnowPlow): void:** ellenőrzi és engedélyezi a hókotró belépését a sávra
- **AddSnow(amount: int): void:** hó hozzáadása, delegál az állapotra
- **ApplySalt(): void:** só alkalmazása, delegál az állapotra
- **FormIce(): void:** jégképződés kiváltása, delegál az állapotra
- **IsPassable(): boolean:** igaz, ha a sáv járható
- **IsSlippery(): boolean:** igaz, ha a sáv csúszós
- **GetState(): LaneState:** visszaadja az aktuális állapot objektumot

- **RegisterPassage(v: Vehicle): void:** regisztrálja az áthaladást, ha elég jármű ment át, jegesedést vált ki
- **CleanWith(plow: SnowPlow): void:** a hókotró fejével elvégzi a takarítást
- **SetBlocked(turns: int): void:** megadott körre blokkolja a sávot ütközés esetén

3.3.16 LaneState

- **Felelősség**

Absztrakt osztály, amely a sáv állapotát reprezentálja. Az összes állapot specifikus viselkedést a konkrét alosztályok valósítják meg.

- **Ősosztályok**

- **Interfészek**

- **Asszociációk**

- **Attribútumok**

- **Metódusok**

- **OnSnowAdded(lane: Lane, amount: int): void:** hó hozzáadásakor hívódik meg, az alosztály elvégzi az állapotváltozást
- **OnSaltApplied(lane: Lane): void:** só alkalmazásakor hívódik
- **OnIceFormed(lane: Lane): void:** jégképződéskor hívódik
- **IsPassable(): boolean:** megadja, hogy a sáv járható-e
- **IsSlippery(): boolean:** megadja, hogy a sáv csúszós-e
- **CanBePushed(): boolean:** megadja, hogy söprő/hányó fejjel eltávolítható-e a sáv tartalma

3.3.17 Node

- **Felelősség**

Absztrakt osztály, amely az úthálózat csomópontjait reprezentálja. Minden csomópont képes visszaadni a hozzá csatlakozó utakat.

- **Ősosztályok**

- **Interfészek**

- **Asszociációk**

- **connectedRoads: List<Road>:** a csomóponthoz csatlakozó utak listája

- **Attribútumok**
 - **connectedRoads: List<Road>**: a csatlakozó utak listája
- **Metódusok**
 - **GetConnectedRoads(): List<Road>**: visszaadja a csomóponthoz csatlakozó utakat

3.3.18 Pathfinder

- **Felelősség**

Az útvonalkeresési logikát tartalmazza. Minden Car saját Pathfinder példánnyal rendelkezik. Meghatározza a legrövidebb utat két csomópont között, visszaadja a szomszédos sávokat, az út másik végpontját és a járható sávokat.

- **Ősosztályok**
-

- **Interfészek**
-

- **Asszociációk**

- **city: City**: a városmodell, amelynek útjain az útvonalkeresés zajlik

- **Attribútumok**

- **city: City**: a városra való hivatkozás az útvonalkereséshez

- **Metódusok**

- **GetShortestPath(from: Node, to: Node): List<Lane>**: visszaadja a legrövidebb útvonalat sávok listájaként a két csomópont között
- **GetNeighborLane(road: Road, lane: Lane, direction: Direction): Lane**: visszaadja a megadott sáv szomszédos sávját az adott irányban
- **GetOtherEnd(road: Road, node: Node): Node**: visszaadja az út másik végpontját a megadott csomópontból nézve
- **GetPassableLanes(road: Road): List<Lane>**: visszaadja az úton lévő járható sávokat

3.3.19 Player

- **Felelősség**

Absztrakt osztály, amely a játékosokat reprezentálja. Minden játékos rendelkezik névvel és körönként végrehajtandó döntési metódussal.

- **Ősosztályok**
-

- **Interfészek**
-

- **Asszociációk**
-

- **Attribútumok**
 - **name: String:** a játékos neve
- **Metódusok**
 - **TakeTurn(): void:** a játékos meghozza a kör döntéseit
 - **GetName(): String:** visszaadja a játékos nevét

3.3.20 Road

- **Felelősség**

Egy irányított utat modellez két csomópont között. Tárolja a sávok listáját és a két végpontot. Felelős a sávjai kezeléséért és a hóesés továbbításáért.

- **Ősosztályok**
-

- **Interfészek**
-

- **Asszociációk**

- **from: Node:** az út kiindulópontja
- **to: Node:** az út végpontja
- **lanes: List<Lane>:** az úton lévő sávok

- **Attribútumok**

- **from: Node:** az út kezdő csomópontja
- **to: Node:** az út záró csomópontja
- **lanes: List<Lane>:** az úton lévő sávok listája

- **Metódusok**

- **GetLanes(): List<Lane>:** visszaadja az út összes sávját
- **ApplySnow(amount: int): void:** továbbítja a hóesést az összes sávra

3.3.21 SaltHead

- **Felelősség**

Sósóró hókotró fej, amely sót szór a sávra. Só készlete véges, a takarító töltheti fel.

- **Ősosztályok**

CleanerHead

- **Interfészek**
-

- **Asszociációk**
-

- **Attribútumok**

- **salt: int:** a maradék só készlet mennyisége

- **Metódusok**

- **Clean(lane: Lane): void:** sót szór a sávra, ha van elegendő só
- **IsOperational(): boolean:** igaz, ha a só készlet nem üres
- **Refuel(amount: int): void:** feltölti a só készletet a adott mennyiséggel

3.3.22 SnowPlow

- **Felelősség**

Egy hókotrót modellez, amelyet a CleanerPlayer irányít. A takarító utasítására bizonyos sávra mozog és elvégzi a takarítást a felszerelt fejjel.

- **Ősosztályok**

- **Interfészek**

- **Asszociációk**

- **currentLane: Lane:** az aktuális sáv, amelyen a hókotró áll
- **head: CleanerHead:** a felszerelt hókotró fej
- **owner: CleanerPlayer:** a tulajdonos takarító játékos

- **Attribútumok**

- **currentLane: Lane:** az aktuális pozíció
- **head: CleanerHead:** az aktuálisan felszerelt hókotró fej
- **owner: CleanerPlayer:** a tulajdonos takarító

- **Metódusok**

- **Step(): void:** végrehajtja a körét, a takarító utasítása alapján mozog és takarít
- **MoveTo(lane: Lane): void:** a hókotrót áthelyezi a megadott sávra
- **ChangeHead(newHead: CleanerHead): void:** kicseréli a felszerelt kotrófejet
- **GetCurrentLane(): Lane:** visszaadja az aktuális sávot
- **GetHead(): CleanerHead:** visszaadja az aktuálisan felszerelt fejet
- **ExecuteClean(): void:** elvégzi a takarítást az aktuális sávon a felszerelt fejjel

3.3.23 SweepHead

- **Felelősség**

Söprő hókotró fej, amely a havat és a feltört jeget a jobbra szomszédos sávba tolja. Ha a hókotró a jobb szélső sávban van, az út mellé söpri. Jegen nem működik. Nem igényel üzemanyagot.

- **Ősosztályok**

CleanerHead

- **Interfészek**

- **Asszociációk**

- **Attribútumok**

- **Metódusok**

- **Clean(lane: Lane): void:** a havat/feltört jeget a szomszédos sávba tolja, a szomszéd hóvastagsága megnő
- **IsOperational(): boolean:** mindig igaz

3.3.24 Terminal

- **Felelősség**

Egy buszos végállomást modellez, amely a buszok útvonalának egyik végpontja. Értesítést fogad, amikor egy busz megérkezik hozzá. Csomópontként illeszkedik az úthálózatba.

- **Ősosztályok**

Node

- **Interfészek**

- **Asszociációk**

- **connectedRoads:** a végállomáshoz csatlakozó utak (örökölt)

- **Attribútumok**

- **connectedRoads: List<Road>:** a csomóponthoz csatlakozó utak (örökölt)

- **Metódusok**

- **GetConnectedRoads(): List<Road>:** visszaadja a csomóponthoz csatlakozó utakat (örökölt)
- **NotifyArrival(bus: Bus): void:** értesítést fogad egy busz érkezéséről

3.3.25 ThickSnowState

- **Felelősség**

A vastag hóval borított sáv állapotot reprezentálja. Járművek számára elakadást okoz, de hókotró számára járható. Sóval és hókotró fejekkel eltávolítható.

- **Ősosztályok**

LaneState

- **Interfészek**

- **Asszociációk**

- **Attribútumok**
 - **snowThickness: int:** a hóréteg vastagsága
- **Metódusok**
 - **OnSnowAdded(lane: Lane, amount: int): void:** növeli a hóvastagságot
 - **OnSaltApplied(lane: Lane): void:** só hatására a hó olvad, állapota ClearState-be vált
 - **OnIceFormed(lane: Lane): void:** vastag hóban jégképződés nem értelmezett
 - **IsPassable(): boolean:** hamis, járművek elakadnak
 - **IsSlippery(): boolean:** hamis
 - **CanBePushed(): boolean:** igaz, söprő, hányó és sárkányfej eltávolítja

3.3.26 ThinSnowState

- **Felelősség**

A vékony hóval borított sáv állapotot reprezentálja. Átjárható és nem csúszós. További hó hatására vastag hó állapotba, sok jármű áthaladása után jeges állapotba vált. Sóval olvasztható.

- **Ősosztályok**

LaneState

- **Interfészek**

—

- **Asszociációk**

—

- **Attribútumok**

- **snowThickness: int:** a hóréteg vastagsága

- **Metódusok**

- **OnSnowAdded(lane: Lane, amount: int): void:** növeli a hóvastagságot, ha elér egy határt, ThickSnowState-be vált
- **OnSaltApplied(lane: Lane): void:** só hatására ClearState-be vált
- **OnIceFormed(lane: Lane): void:** elegendő áthaladás után IcyState-be vált
- **IsPassable(): boolean:** igaz
- **IsSlippery(): boolean:** hamis
- **CanBePushed(): boolean:** igaz

3.3.27 ThrowHead

- **Felelősség**

Hányó hókotró fej, amely a havat és a feltört jeget minden esetben az út mellé szórja (nem a szomszéd sávba). Jegen nem működik. Nem igényel üzemanyagot.

- **Ősosztályok**

CleanerHead

- **Interfészek**

- **Asszociációk**

- **Attribútumok**

- **Metódusok**

- **Clean(lane: Lane): void:** a havat/feltört jeget az út mellé szórja, a sáv ClearState-be kerül
- **IsOperational(): boolean:** mindig igaz

3.3.28 Vehicle

- **Felelősség**

Absztrakt osztály, amely a járműveket reprezentálja. Minden jármű nyilvántart egy aktuális sávot, képes lépni, más járművekkel ütközni, és blokkolható.

- **Ősosztályok**

- **Interfészek**

- **Asszociációk**

- **currentLane: Lane:** a jármű aktuális sávja

- **Attribútumok**

- **currentLane: Lane:** az aktuálisan foglalt sáv
- **blockedTurns: int:** hány körre van blokkolva a jármű

- **Metódusok**

- **Step(): void:** a jármű végrehajtja a saját lépését
- **MeetVehicle(v: Vehicle): void:** két jármű találkozásakor (jeges sávon) ütközés kezelése
- **IsBlocked(): boolean:** igaz, ha a jármű blokkolt
- **GetCurrentLane(): Lane:** visszaadja az aktuális sávot
- **DecrementBlock(): void:** csökkenti a blokkolás visszaszámlálóját

3.3.29 Workplace

- **Felelősség**

Az autók munkahelyi célállomása az úthálózaton. Csomópontként illeszkedik a hálózatba. Saját logikával nem rendelkezik, azonosítóként szolgál az autók navigációjában.

- **Ősosztályok**

Node

- **Interfészek**

- **Asszociációk**

- **connectedRoads**: a csomóponthoz csatlakozó utak (örökölt)

- **Attribútumok**

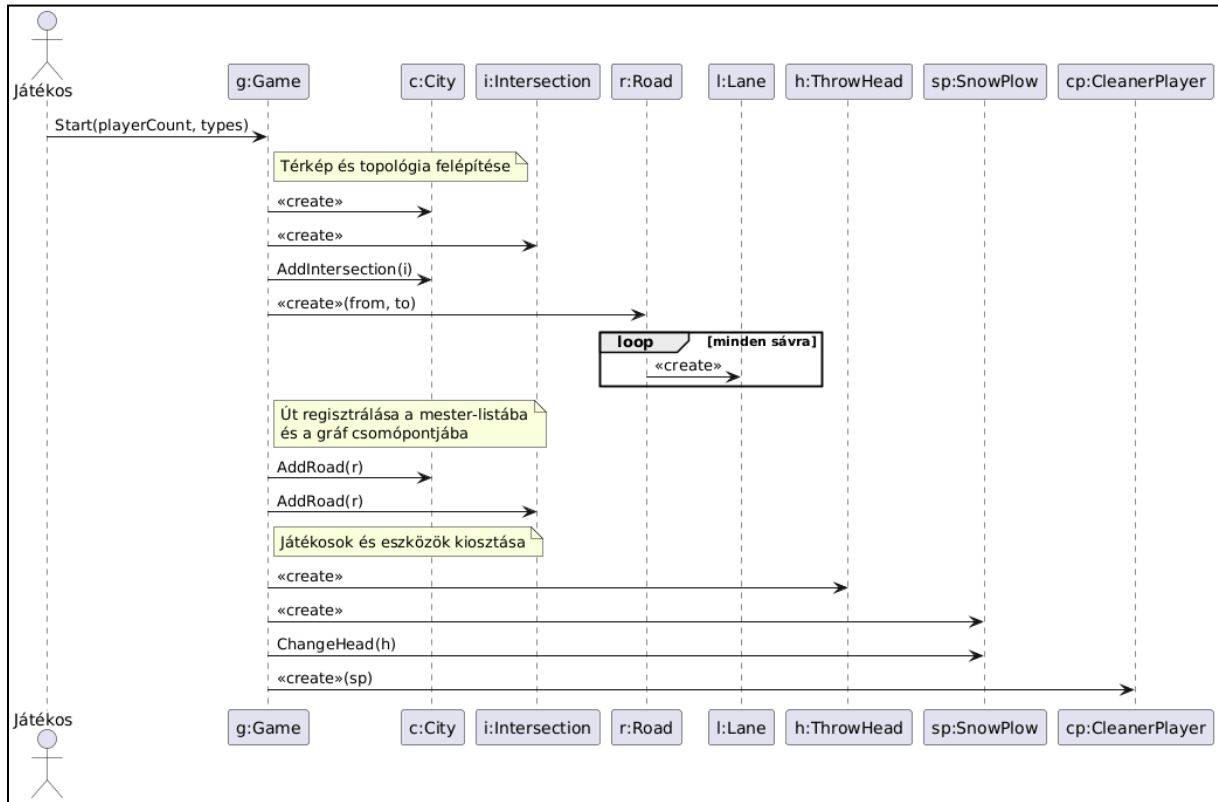
- **connectedRoads: List<Road>**: a csomóponthoz csatlakozó utak (örökölt)

- **Metódusok**

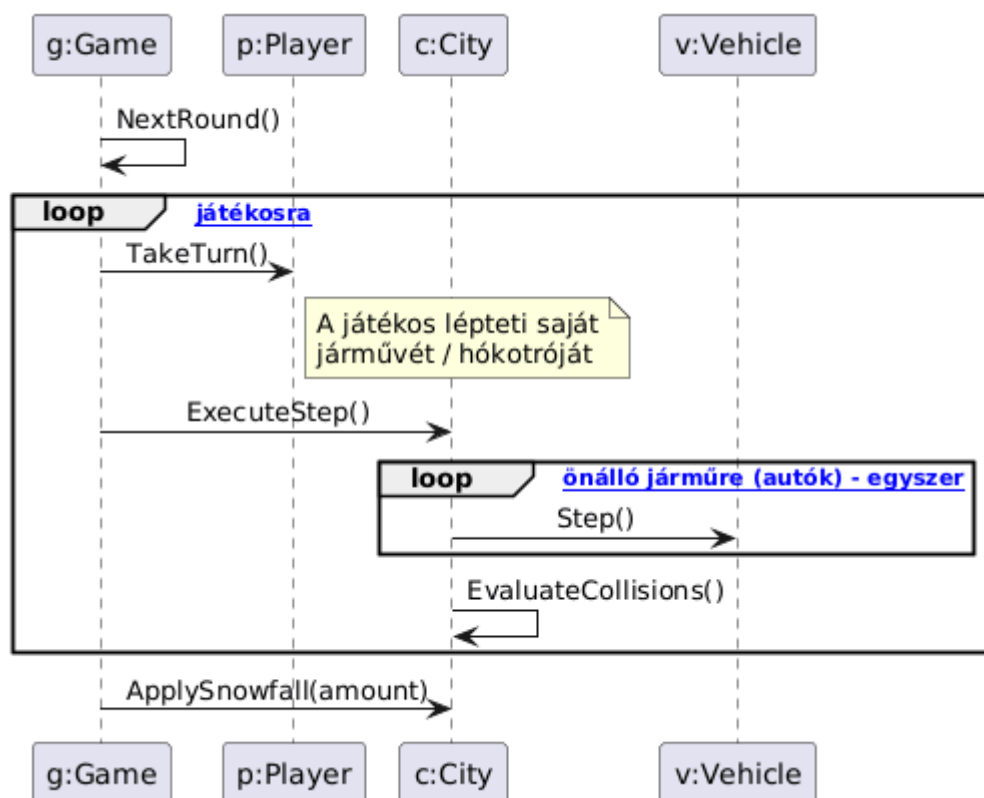
- **GetConnectedRoads(): List<Road>**: visszaadja a csomóponthoz csatlakozó utakat (örökölt)

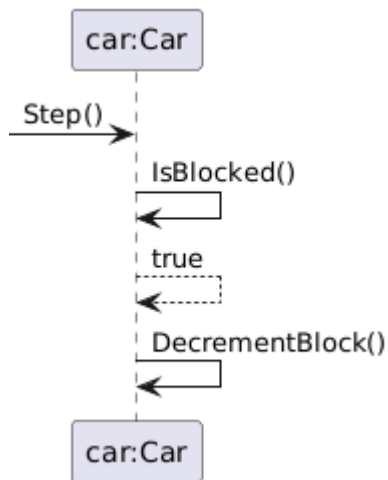
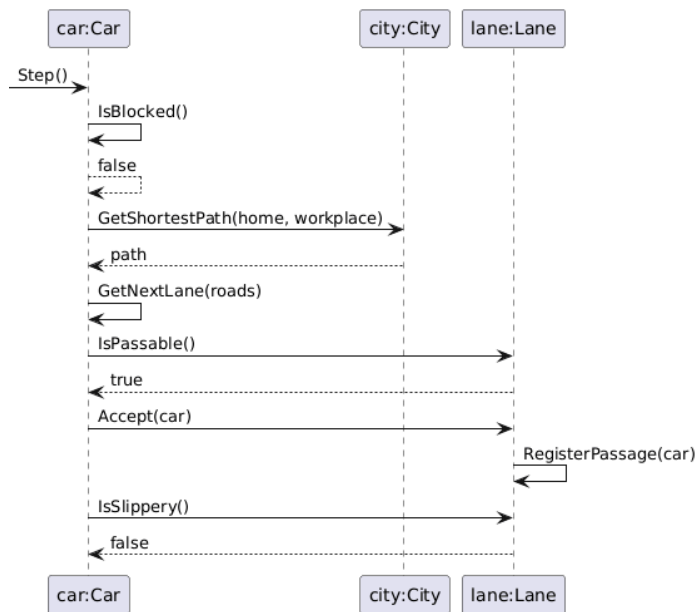
3.4 Szekvencia diagramok

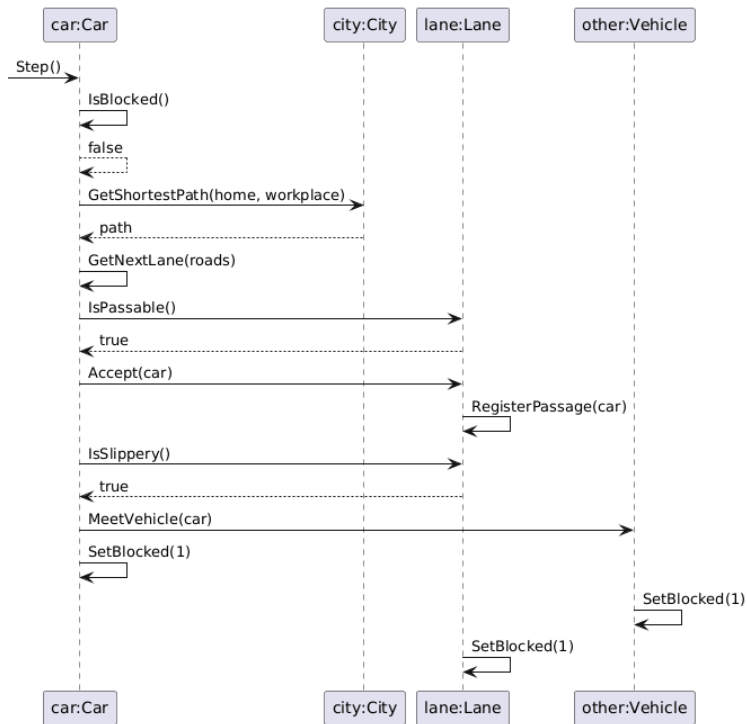
SD-01 – Játék inicializálása



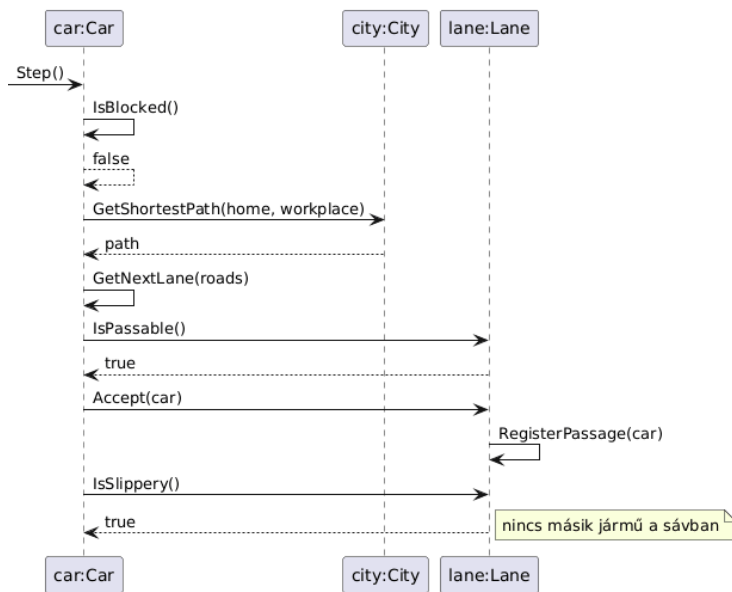
SD-02 – Kör léptetése



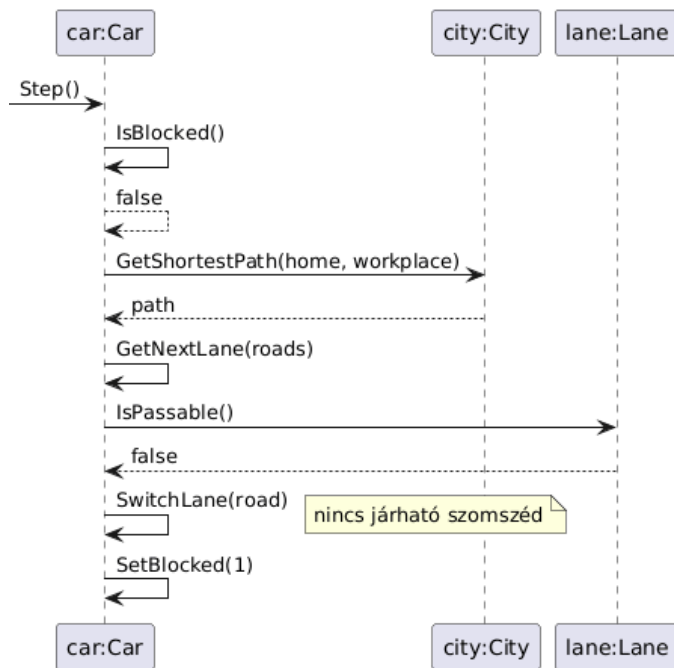
SD-03 – Autó mozgása**SD-03.1 – Autó mozgása - Az autó blokkolt****SD-03.2 – Autó mozgása - Járható út, nem csúszik****SD-03.3 – Autó mozgása - Járható út, csúszik, ütközéses**



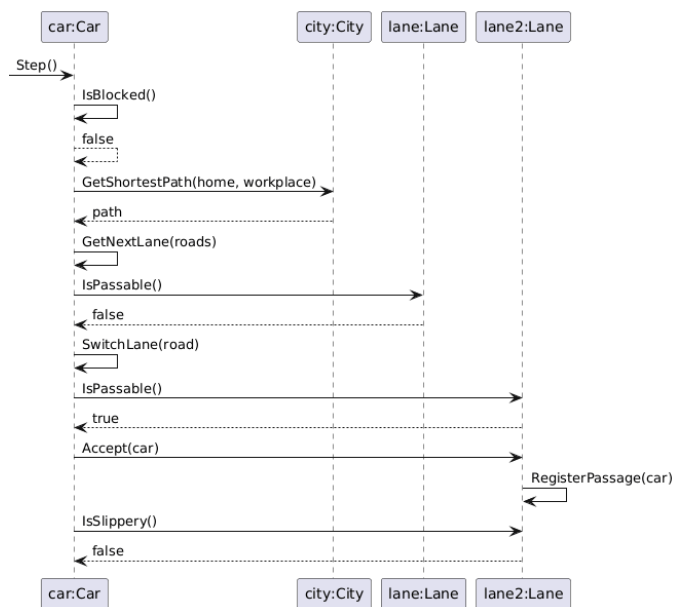
SD-03.4 – Autó mozgása - Járható út, csúszik, nincs ütközés



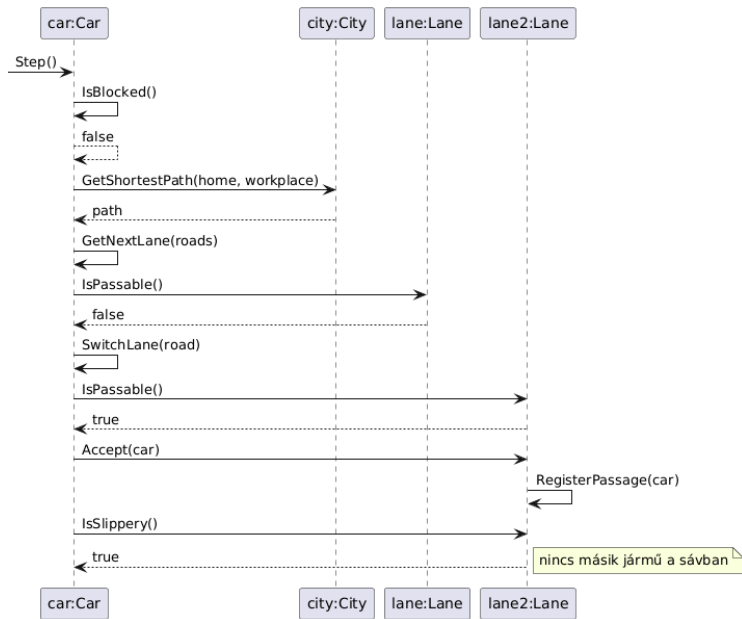
SD-03.5 – Autó mozgása - Nem járható út, nincs járható szomszéd



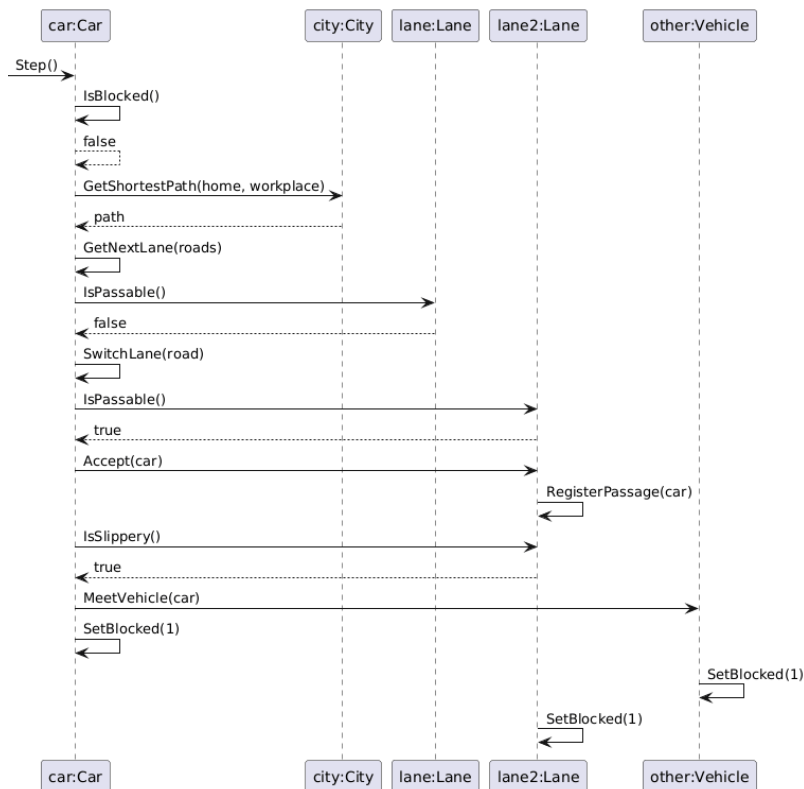
SD-03.6 – Autó mozgása - Nem járható út, járható szomszéd, nem csúszós



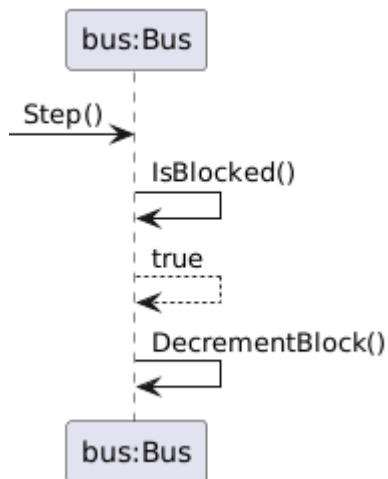
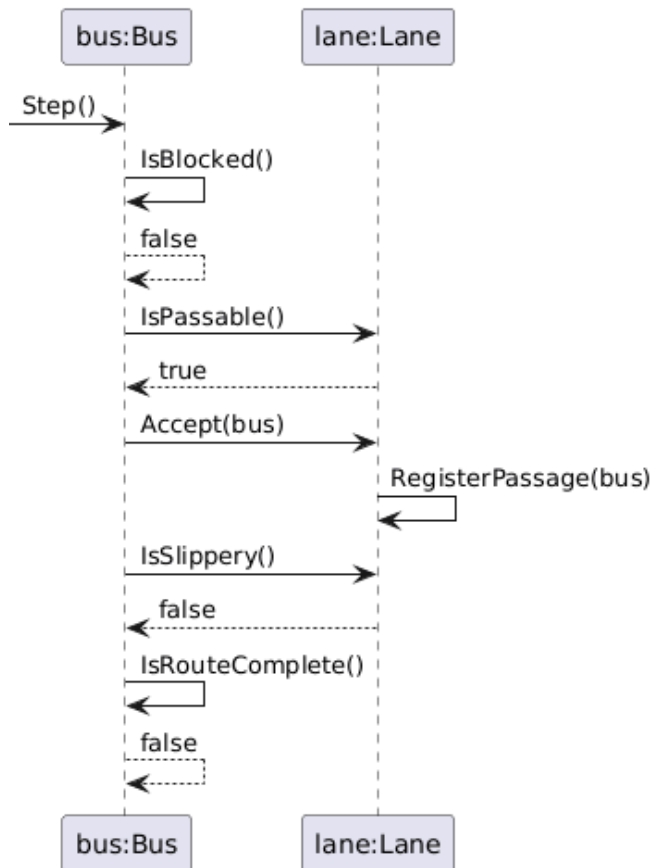
SD-03.7 – Autó mozgása - Nem járható út, járható szomszéd, csúszós, nincs ütközés

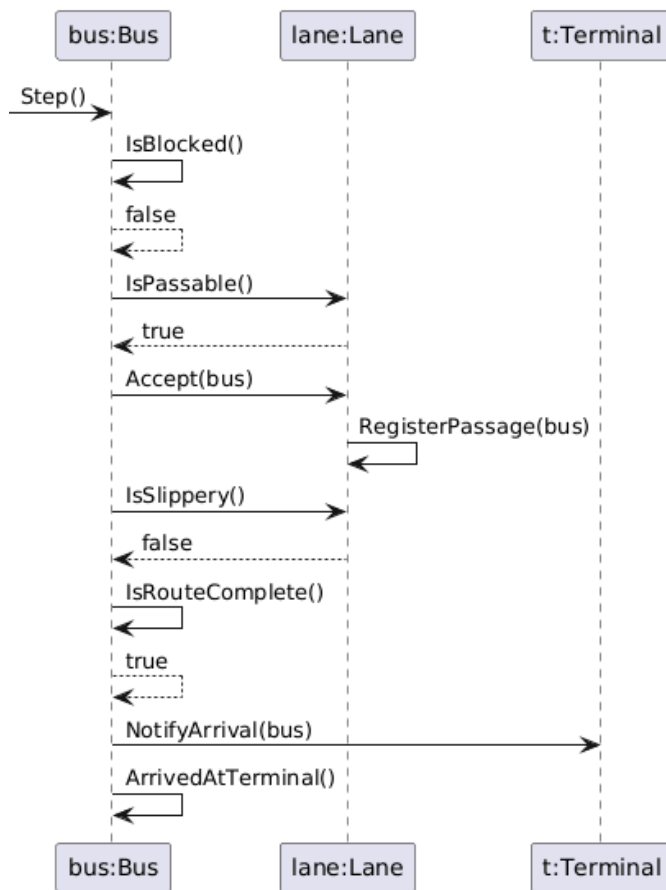


SD-03.8 – Autó mozgása - Nem járható út, járható szomszéd, csúszós, ütközéses

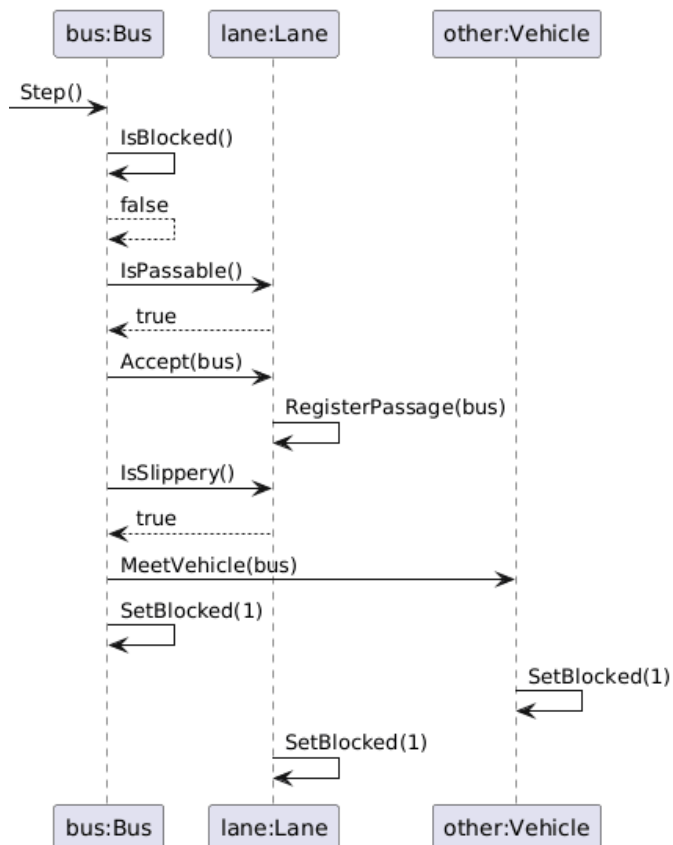


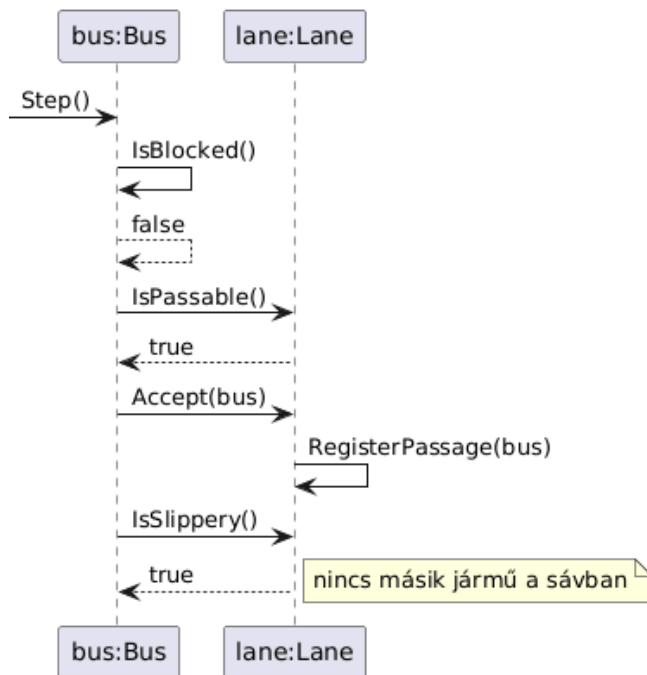
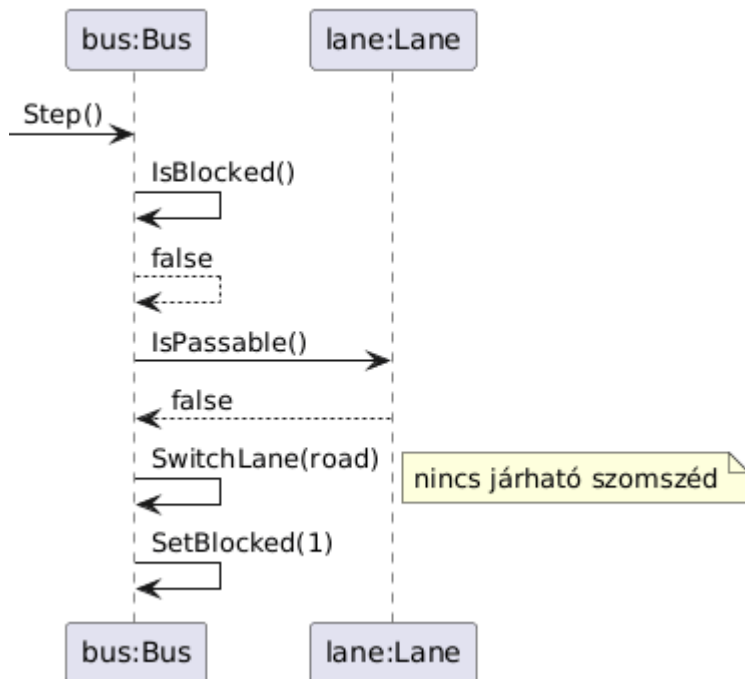
SD-04 – Busz mozgása

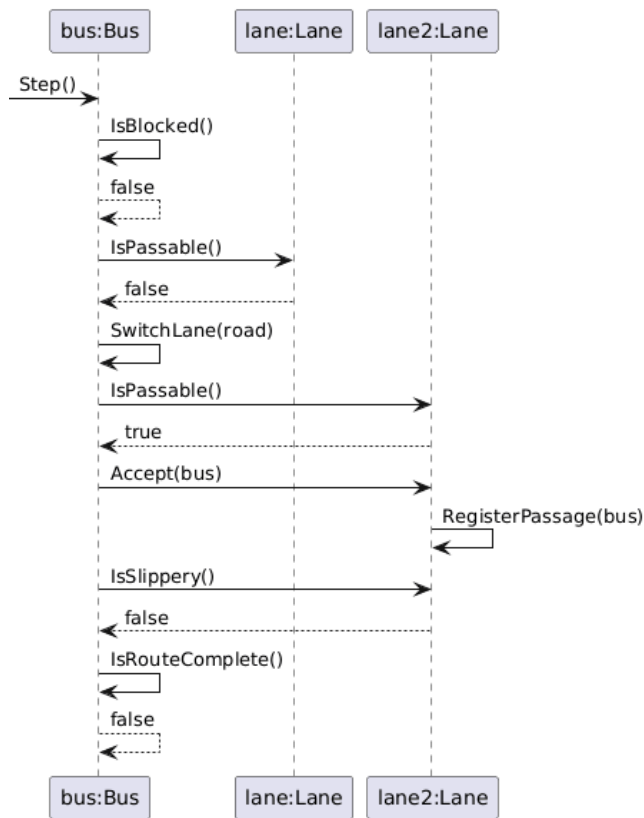
SD-04.1 – Busz mozgása - Busz blokkolt**SD-04.2 – Busz mozgása - Járható út, nem csúszik, nem végállomás****SD-04.3 – Busz mozgása - Járható út, nem csúszik, végállomásra érkezik**



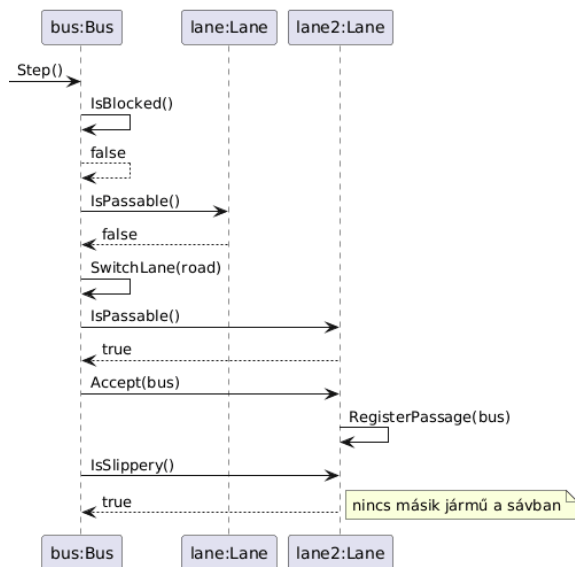
SD-04.4 – Busz mozgása - Járható út, csúszik, ütközéses



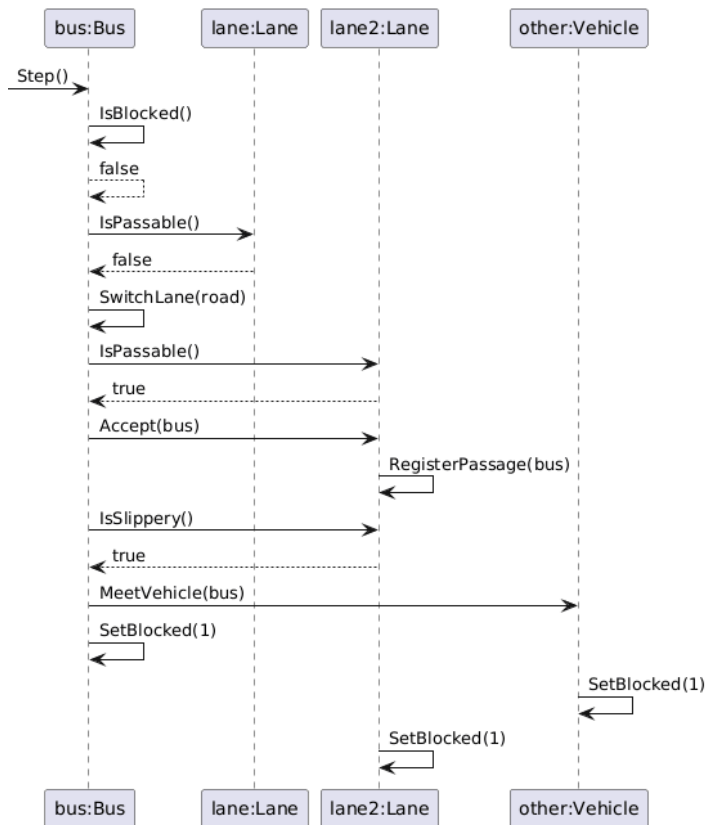
SD-04.5 – Busz mozgása - Járható út, csúszik, nincs ütközés**SD-04.6 – Busz mozgása - Nem járható út, nincs járható szomszéd****SD-04.7 – Busz mozgása - Nem járható út, járható szomszéd, nem csúszós**



SD-04.8 – Busz mozgása - Nem járható út, járható szomszéd, csúszós, nincs ütközés

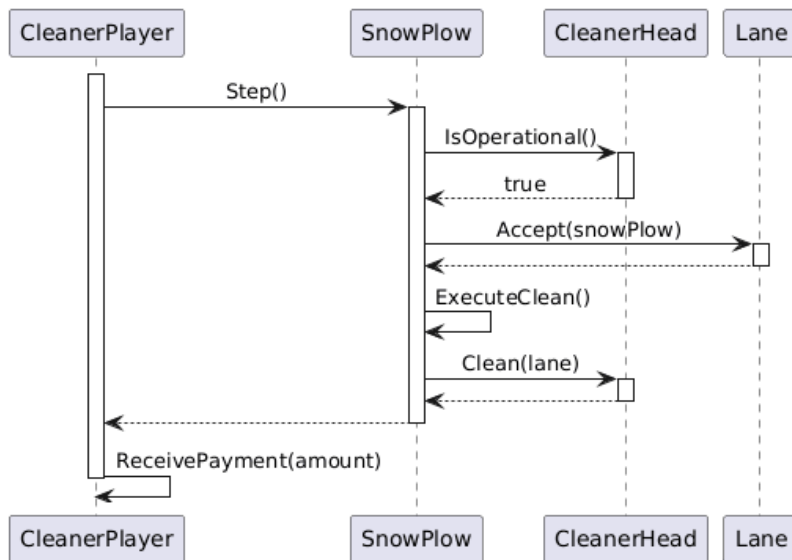


SD-04.9 – Busz mozgása - Nem járható út, járható szomszéd, csúszós, ütközéses

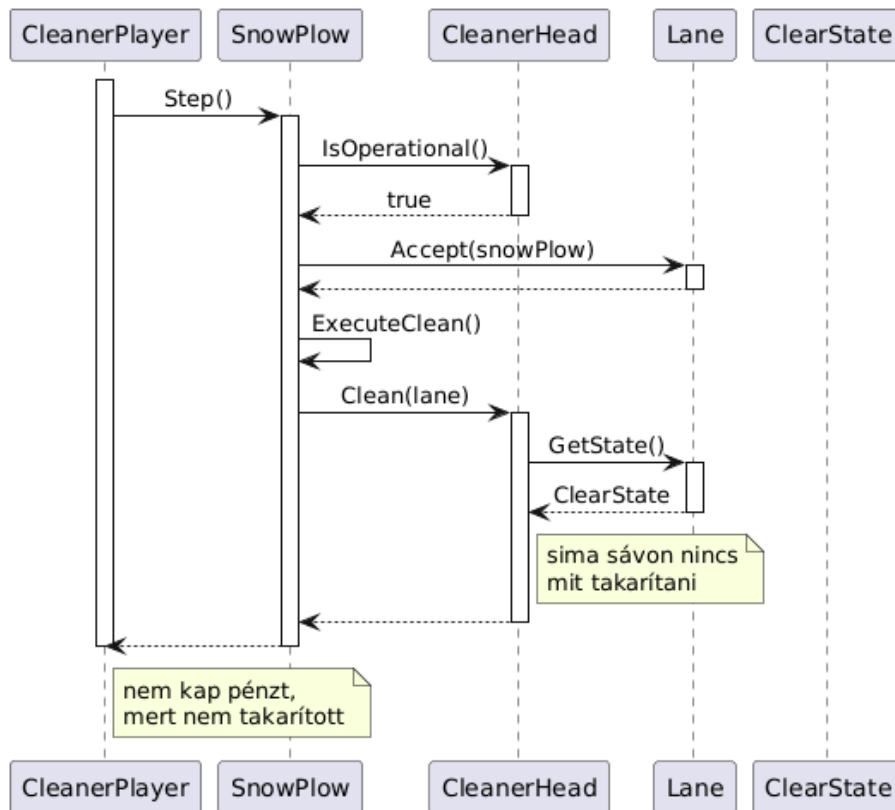


SD-05 – Hókotró

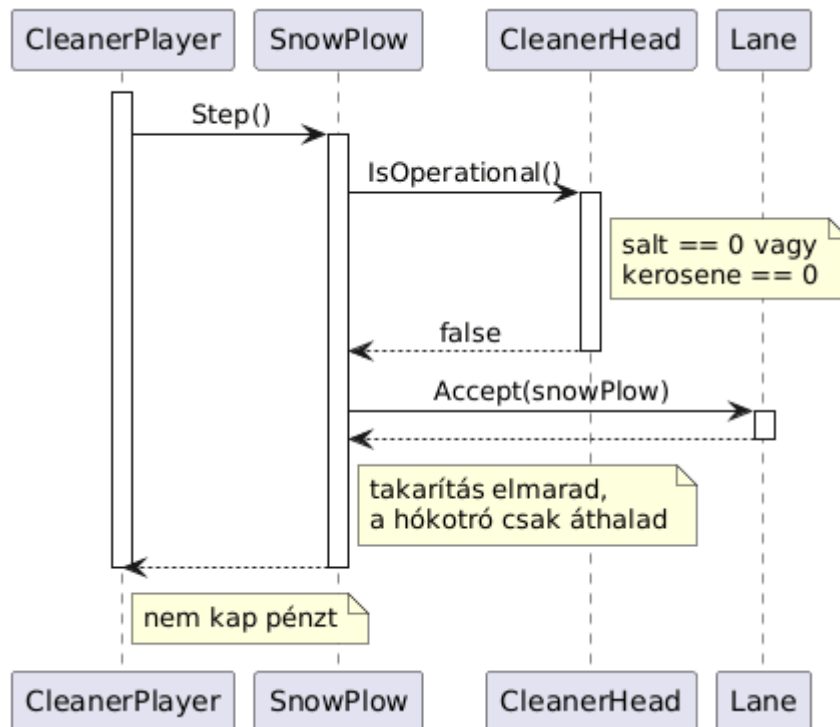
SD-05.1 – Hókotró lépése – normál takarítás (sáv nem sima)



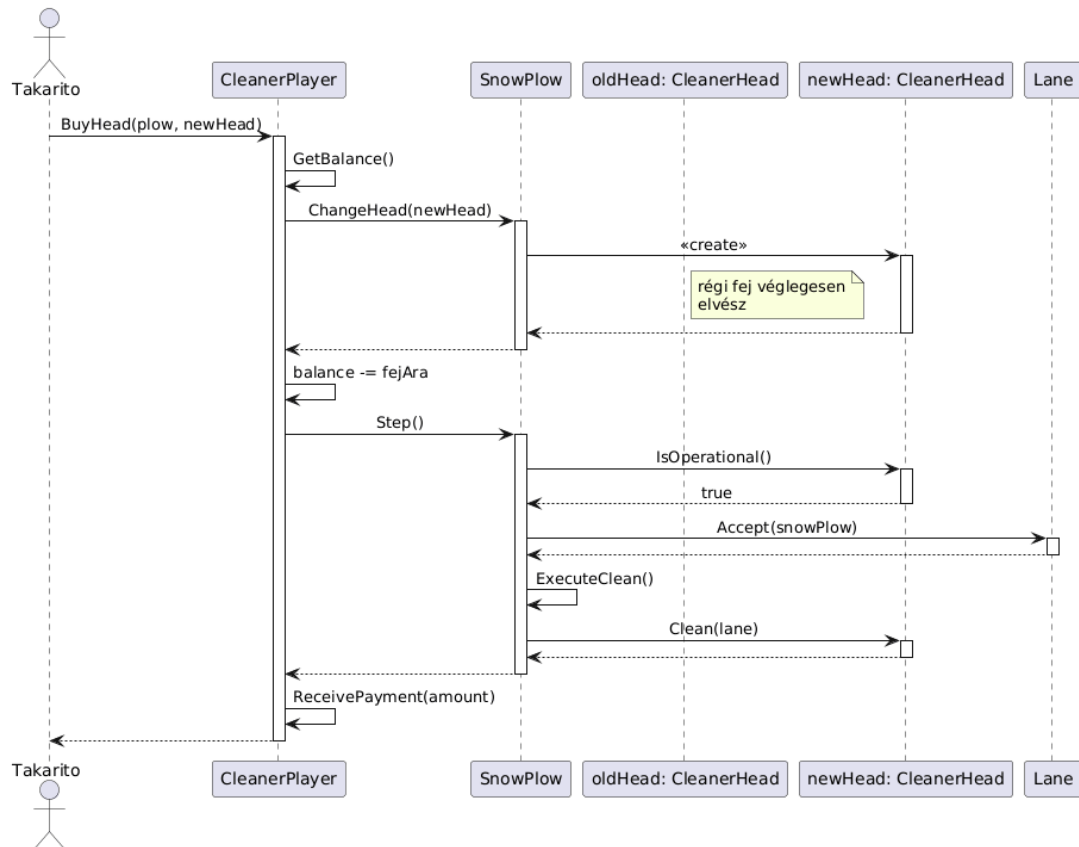
SD-05.2 – Hókotró lépése – célsáv sima (nincs takarítandó)



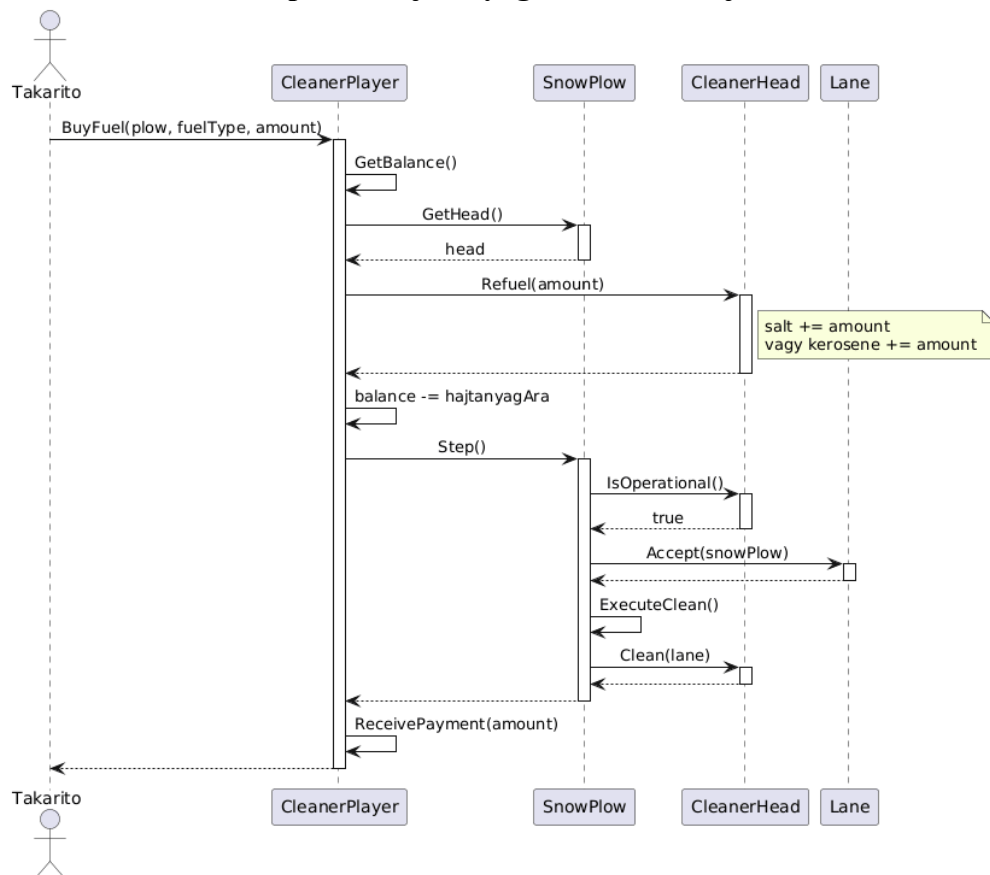
SD-05.3 – Hókotró lépése – fej nem működőképes (kifogyott hajtóanyag)



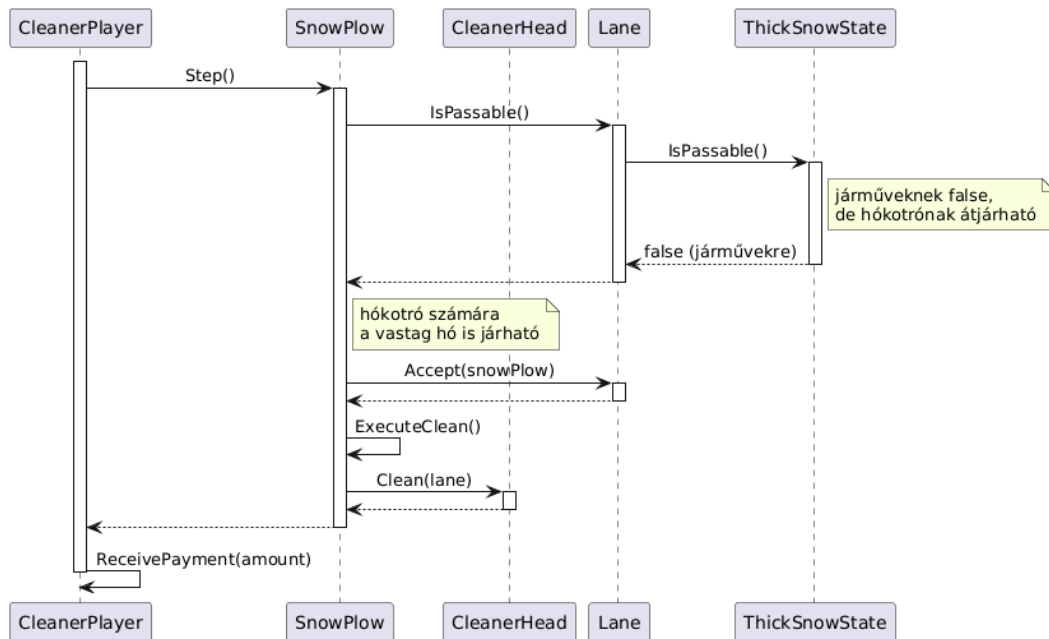
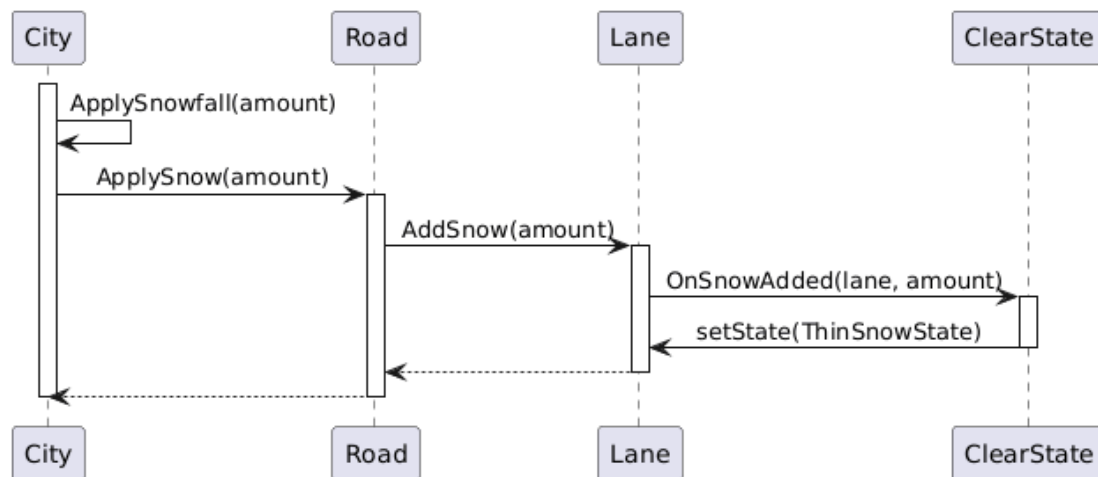
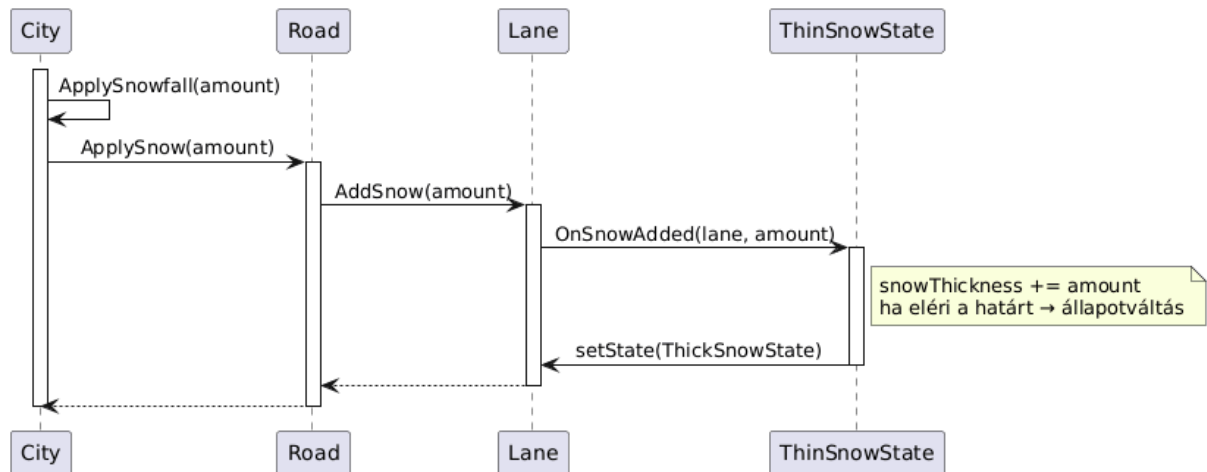
SD-05.4 – Hókotró lépése – fejcsere majd takarítás

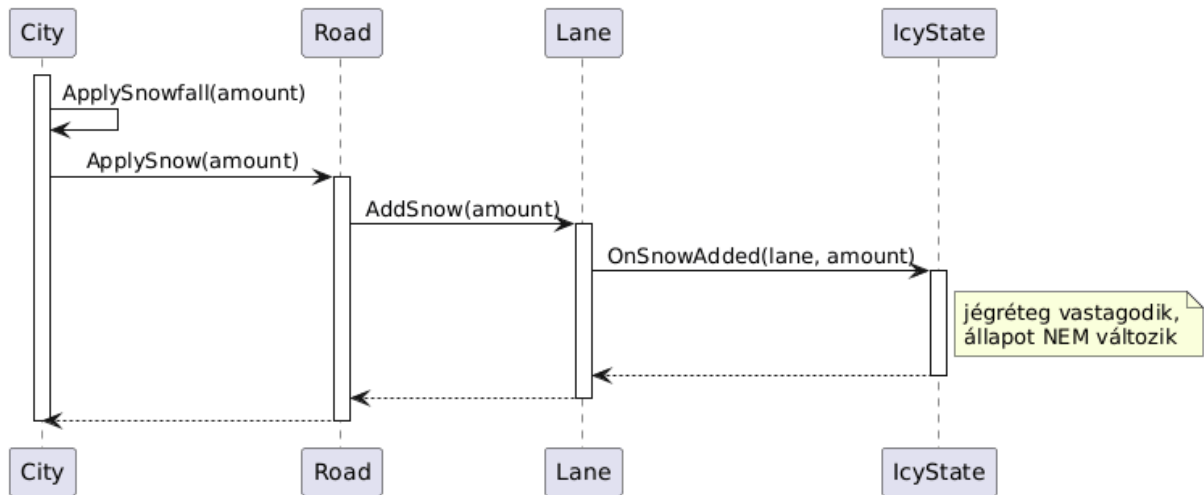


SD-05.5 – Hókotró lépése – hajtóanyag utántöltés majd takarítás

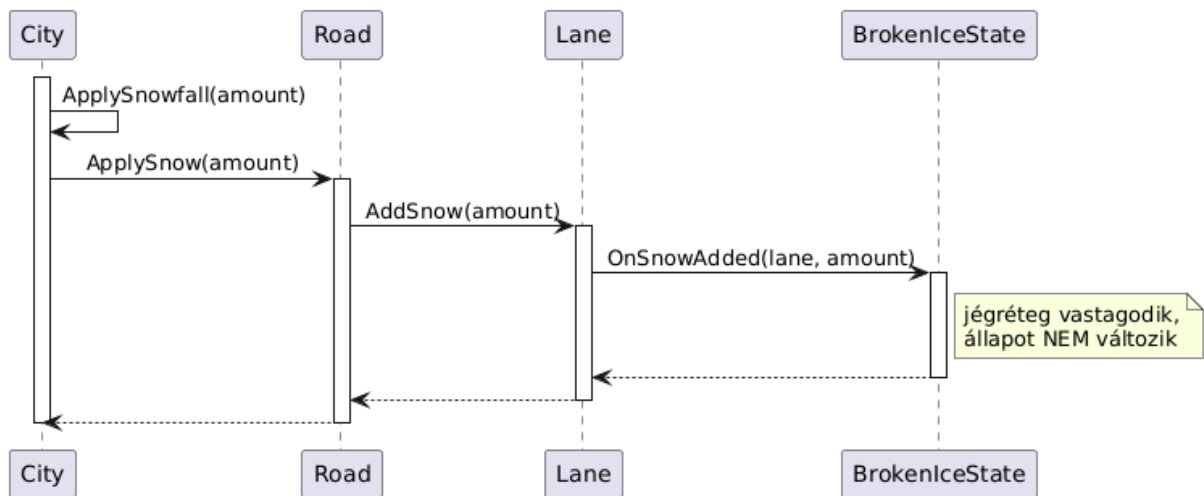


SD-05.6 – Hókotró lépése – vastag hóban is áthalad (hókotró nem akad el)

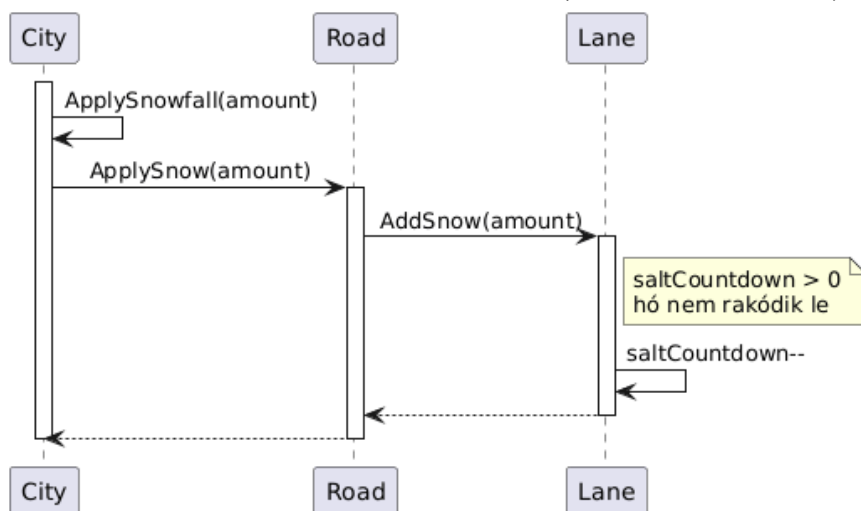
**SD-07 – Havazás****SD-07.1 – Havazás – sima sávra hó esik (ClearState → ThinSnowState)****SD-07.2 – Havazás – vékony havas sávra hó esik (ThinSnowState → ThickSnowState)****SD-07.3 – Havazás – jeges sávra hó esik (IcyState, állapot nem változik)**



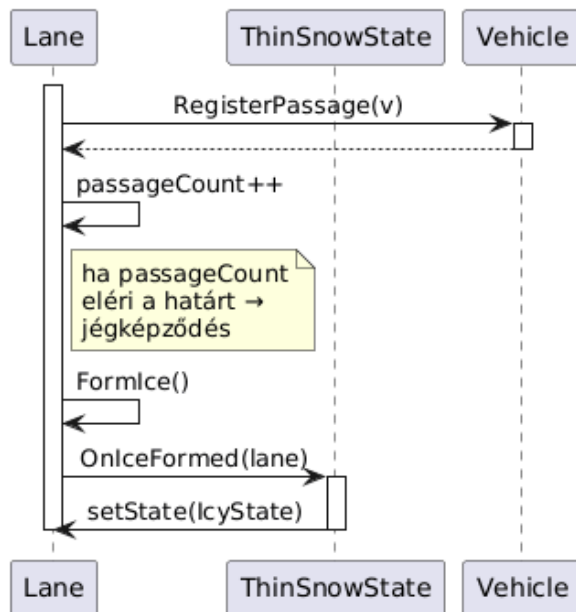
SD-07.4 – Havazás – feltört jeges sávra hó esik (BrokenIceState, állapot nem változik)



SD-07.5 – Havazás – sózott sávra hó esik (saltCountdown véd)

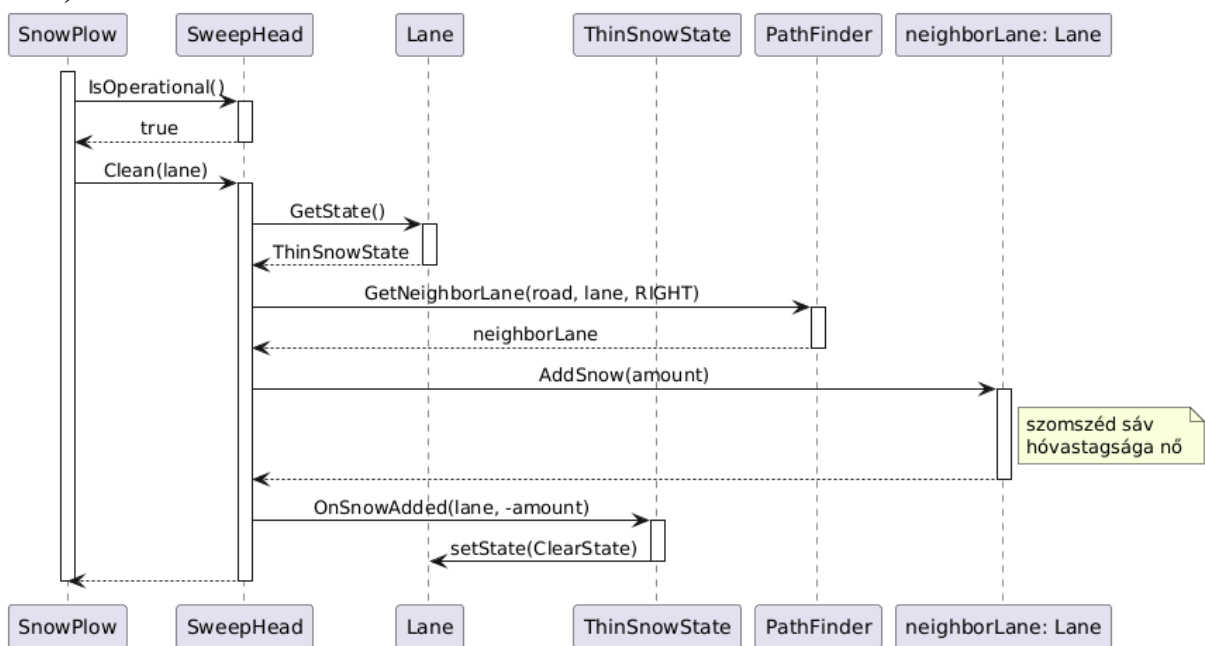


SD-8 – Jégpáncél képződése

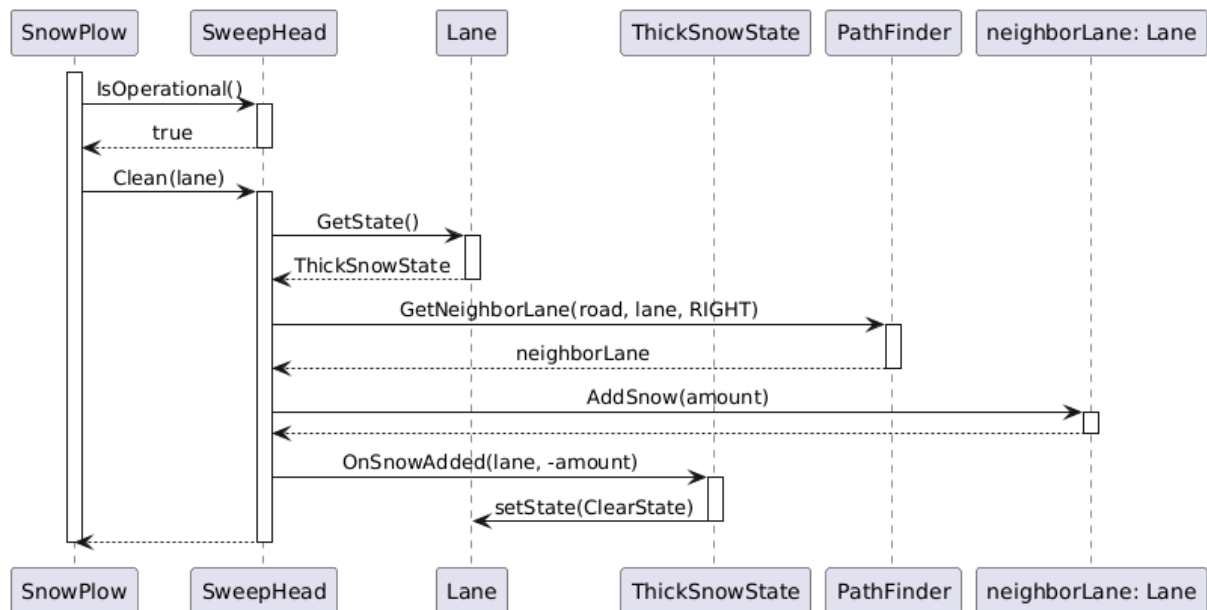


SD-9 – Söprő fejfel takarítás

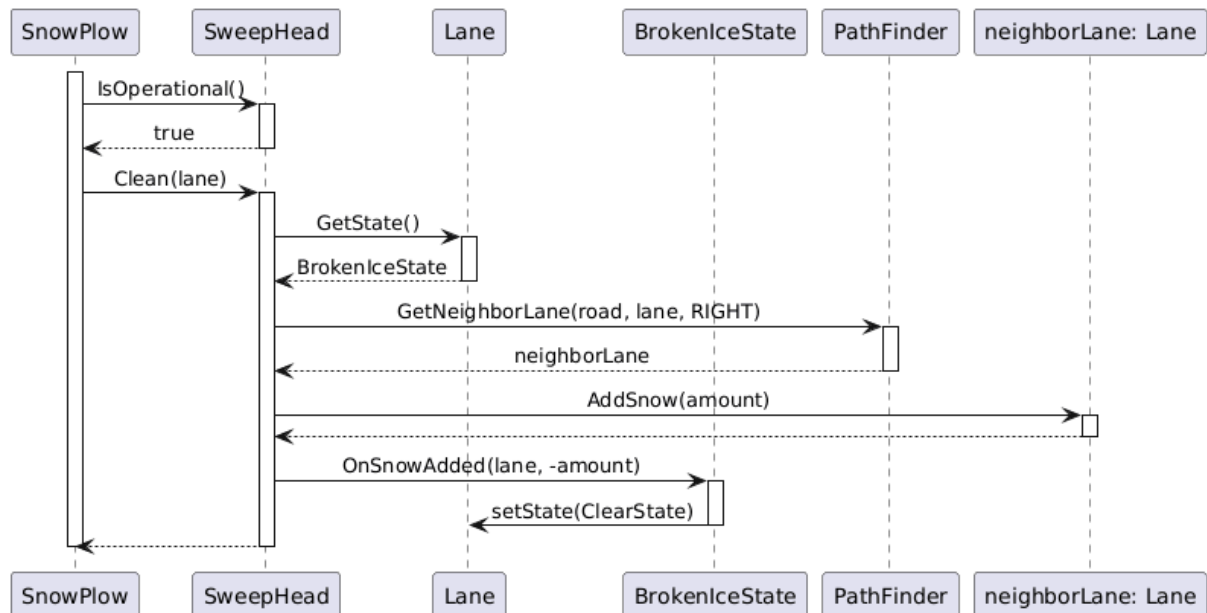
SD-09.1 – Söprő fejfel takarítás – vékony hó (ThinSnowState → ClearState, szomszéd hónő)



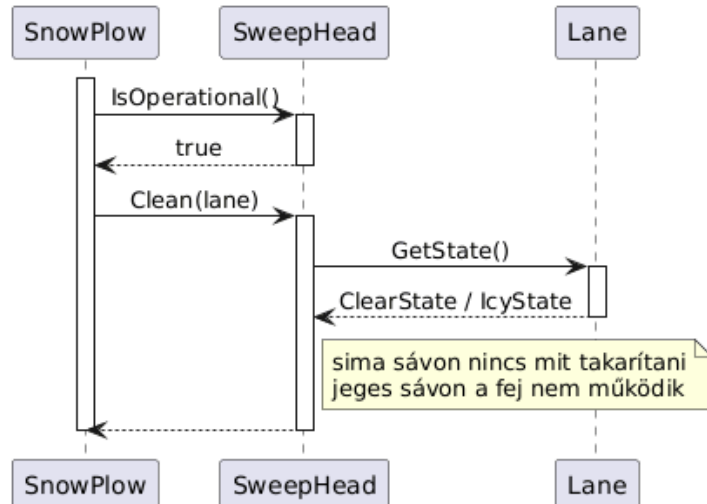
SD-09.2 – Söprő fejfel takarítás – vastag hó (ThickSnowState → ClearState)

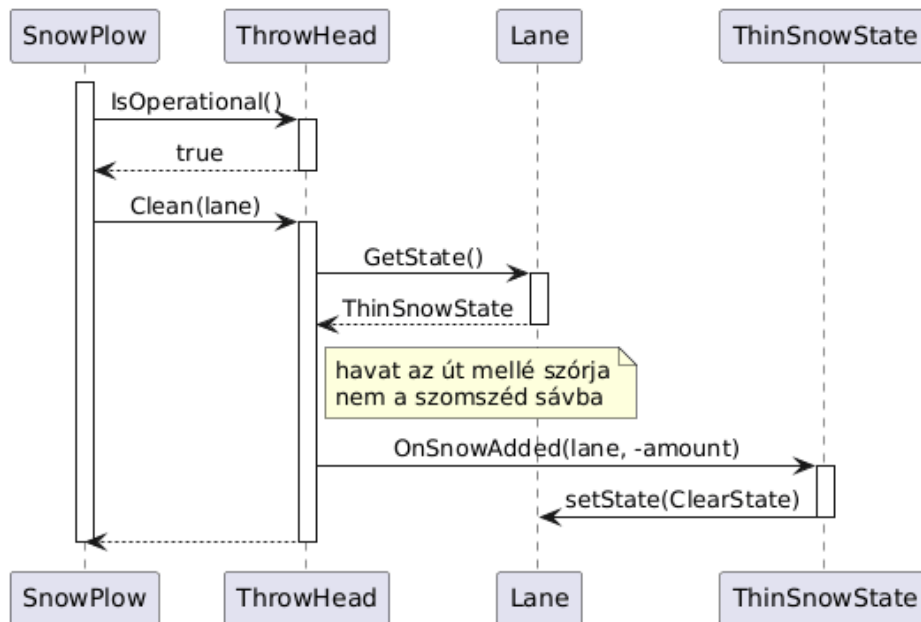
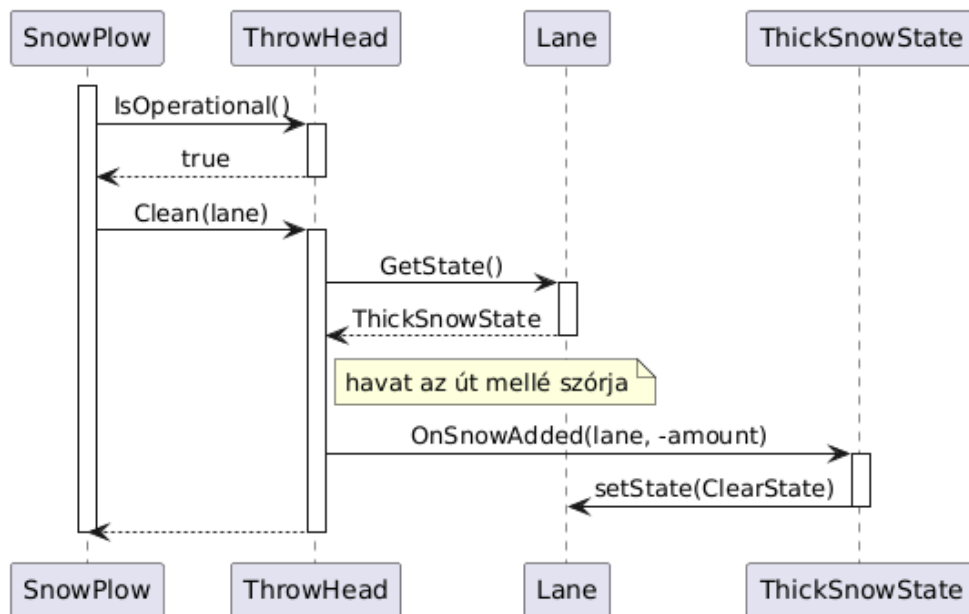


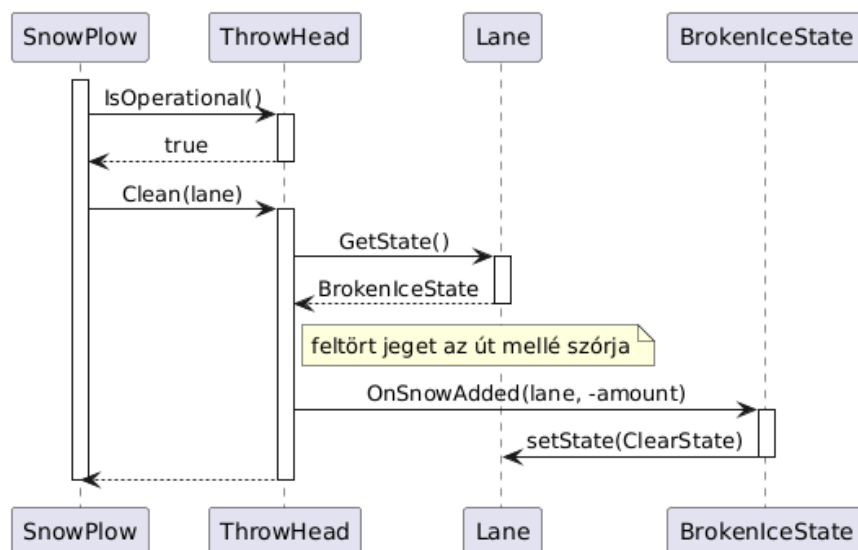
SD-09.3 – Söprő fejfel takarítás – feltört jég (BrokenIceState → ClearState)



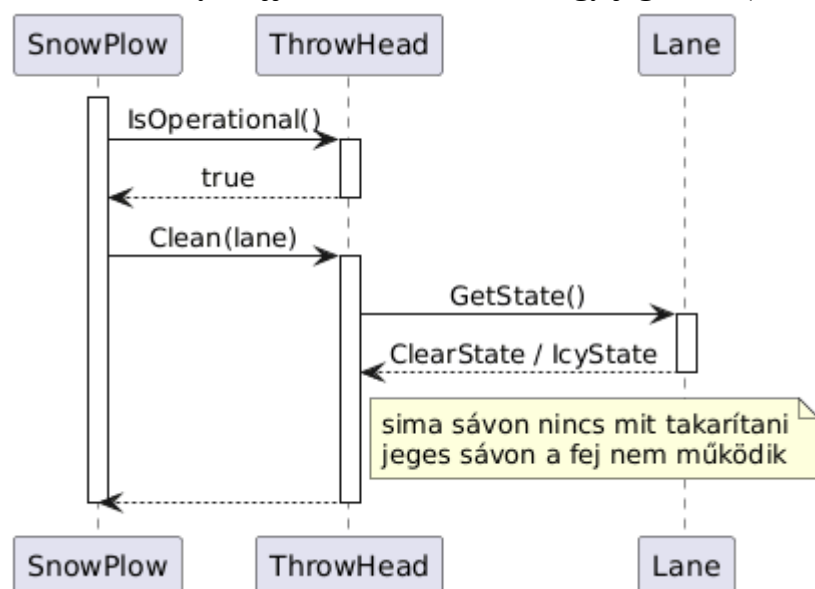
SD-09.4 – Söprő fejfel takarítás – sima vagy jeges sáv (nincs hatás)



SD-10 – Hányó fejjel takarítás**SD-10.1 – Hányó fejjel takarítás – vékony hó (→ ClearState, út mellé szórja)****SD-10.2 – Hányó fejjel takarítás – vastag hó (→ ClearState)****SD-10.3 – Hányó fejjel takarítás – feltört jég (→ ClearState)**

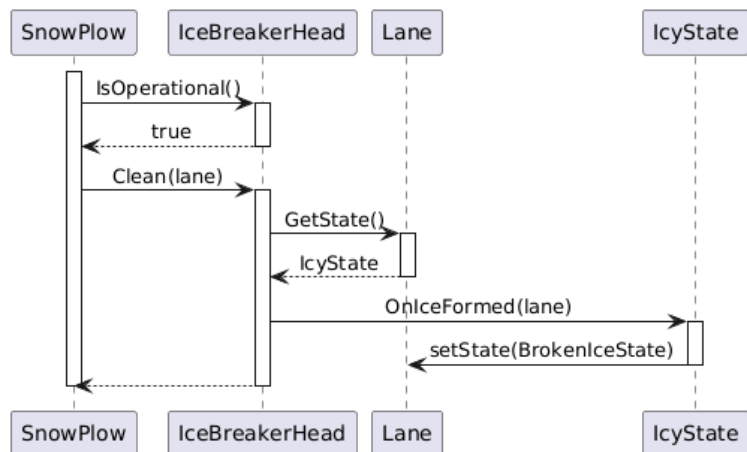


SD-10.4 – Hányó fejjel takarítás – sima vagy jeges sáv (nincs hatás)

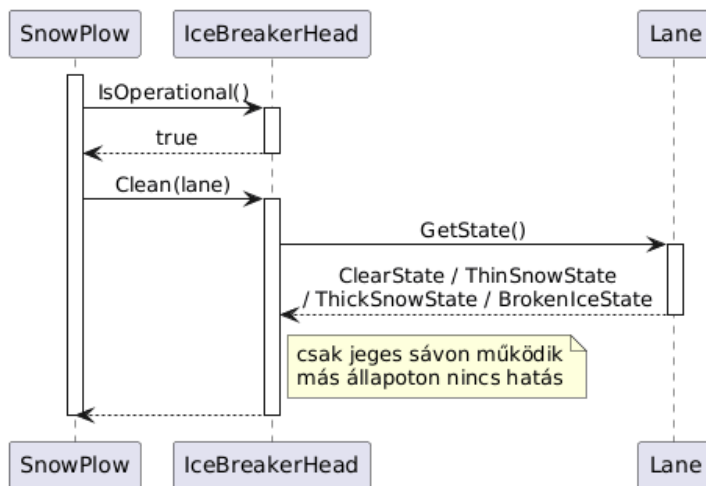


SD-12 – Jégtörő fejjel takarítás

SD-12.1 – Jégtörő fejjel takarítás – jeges sáv (IcyState → BrokenIceState)

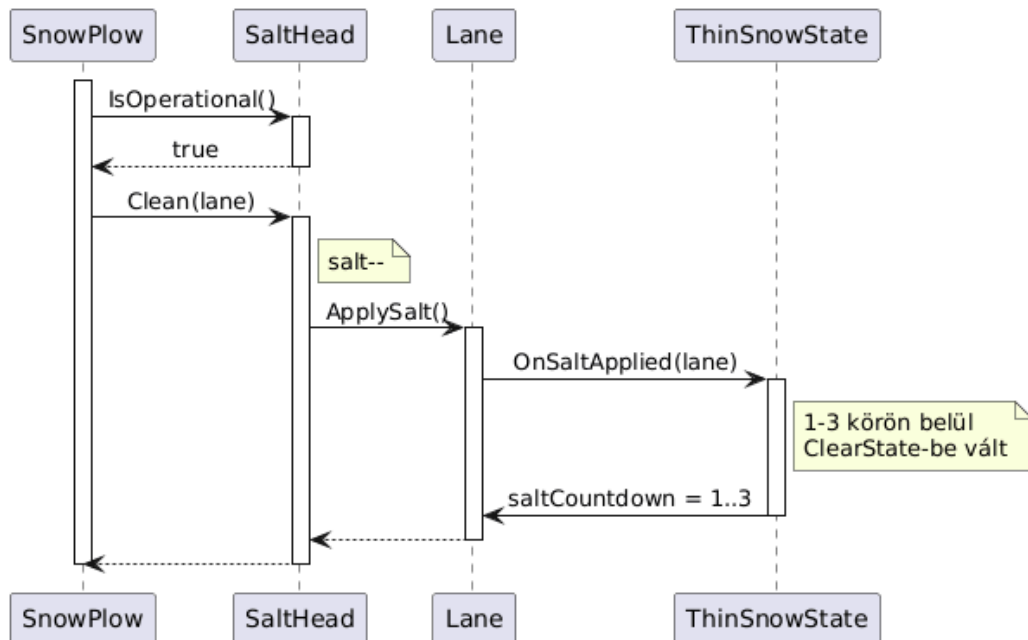


SD-12.2 – Jégtörő fejjel takarítás – nem jeges sáv (nincs hatás)

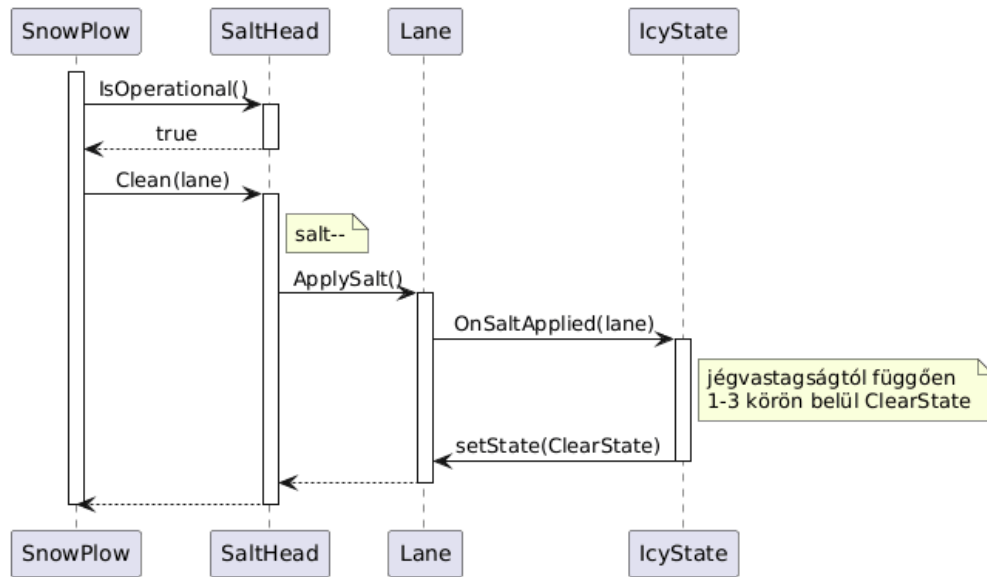


SD-13 – Sósórózó fejfel szózás

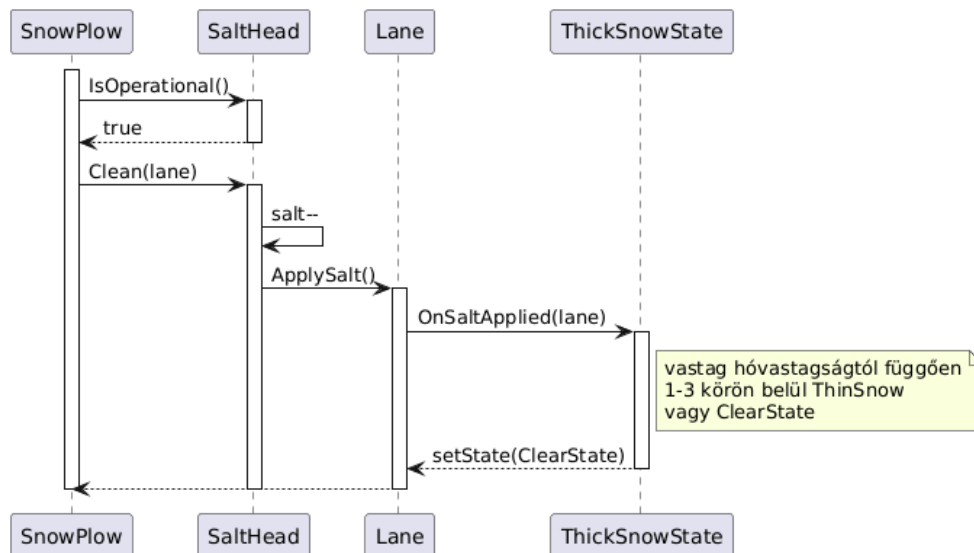
SD-13.1 – Sósórózó fejfel szózás – vékony havas sáv (→ ClearState idővel)



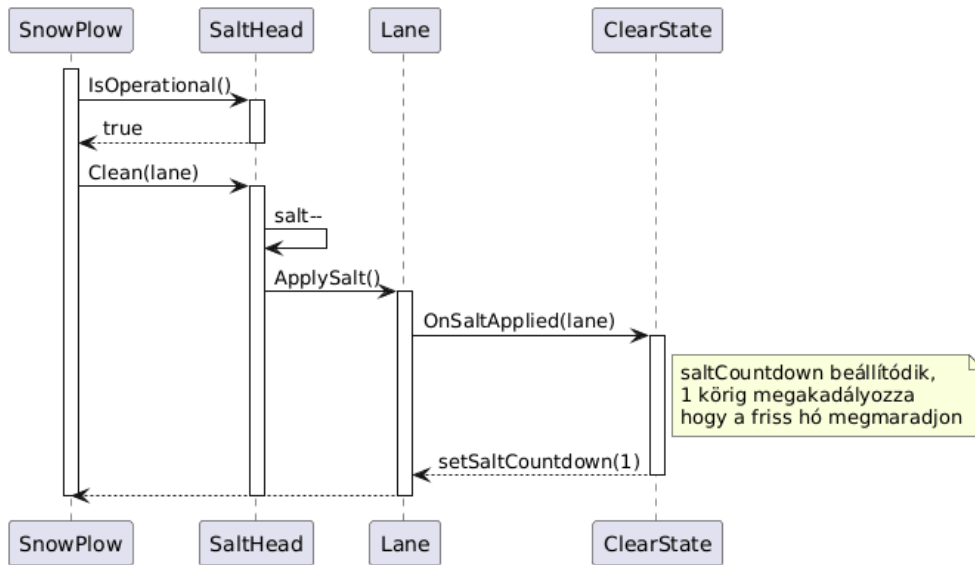
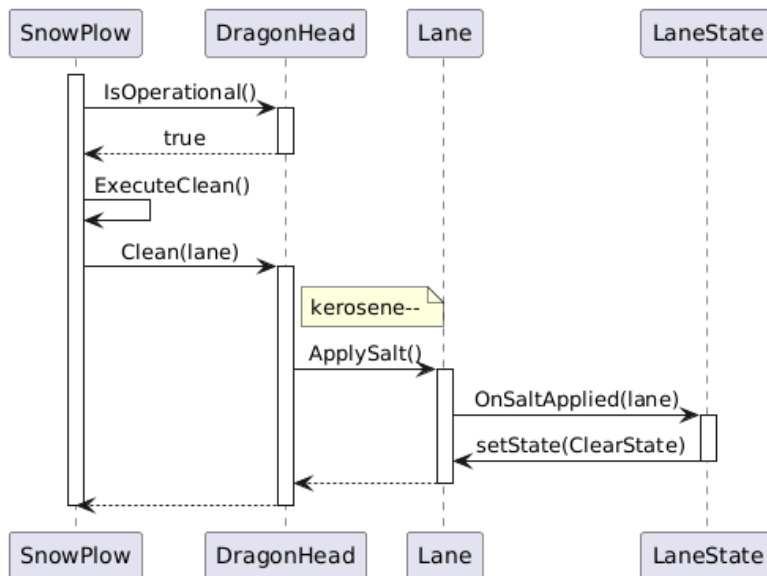
SD-13.2 – Sósórózó fejfel szózás – jeges sáv (→ ClearState idővel)



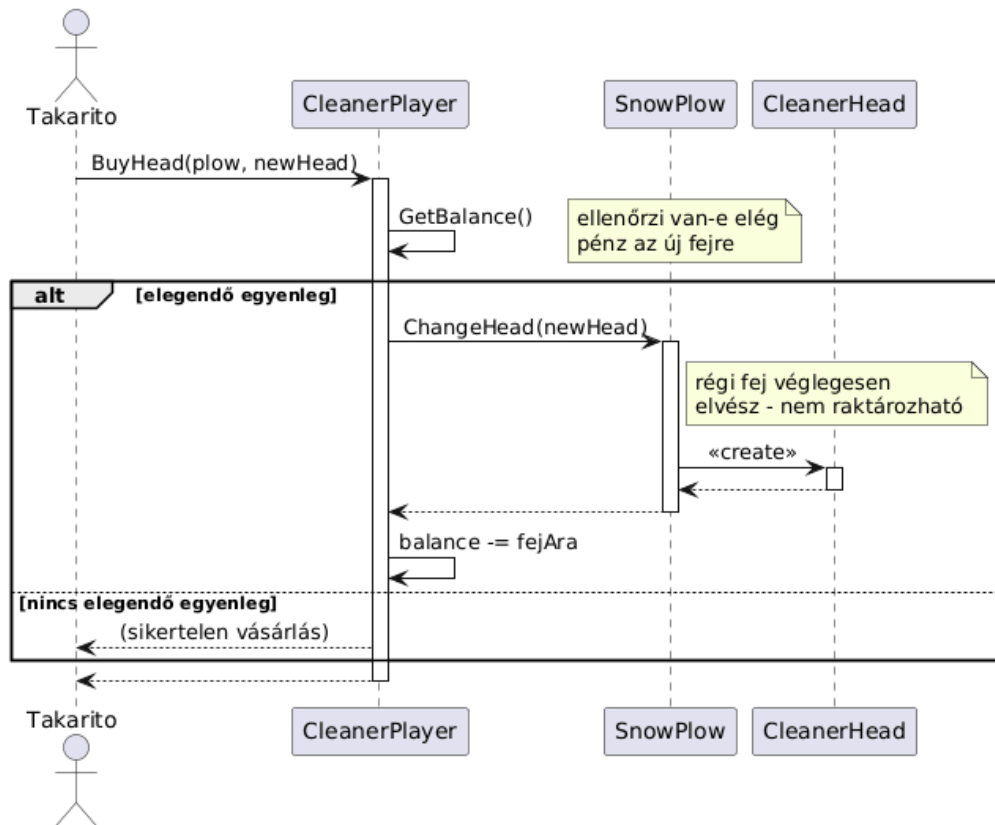
SD-13.3 – Sósóró fejfel sózás – vastag havas sáv



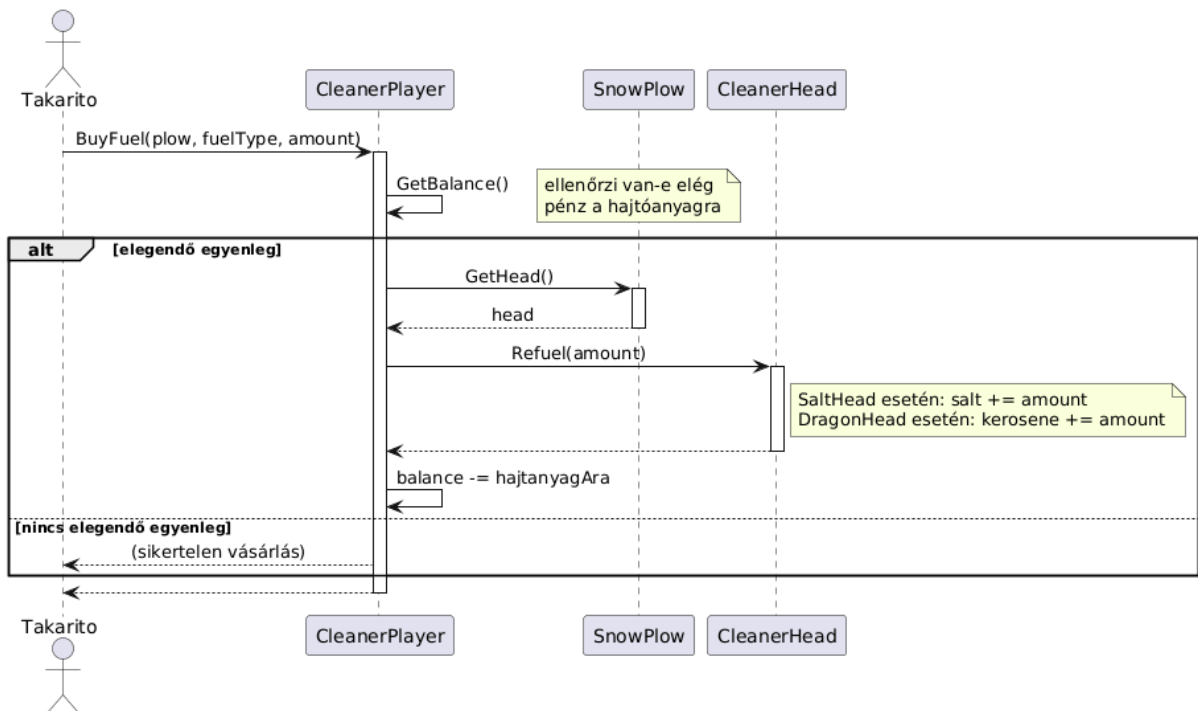
SD-13.4 – Sósóró fejfel sózás – sima sáv (jégképződéstől véd)

**SD-14 – Sárkányfejjel takarítás**

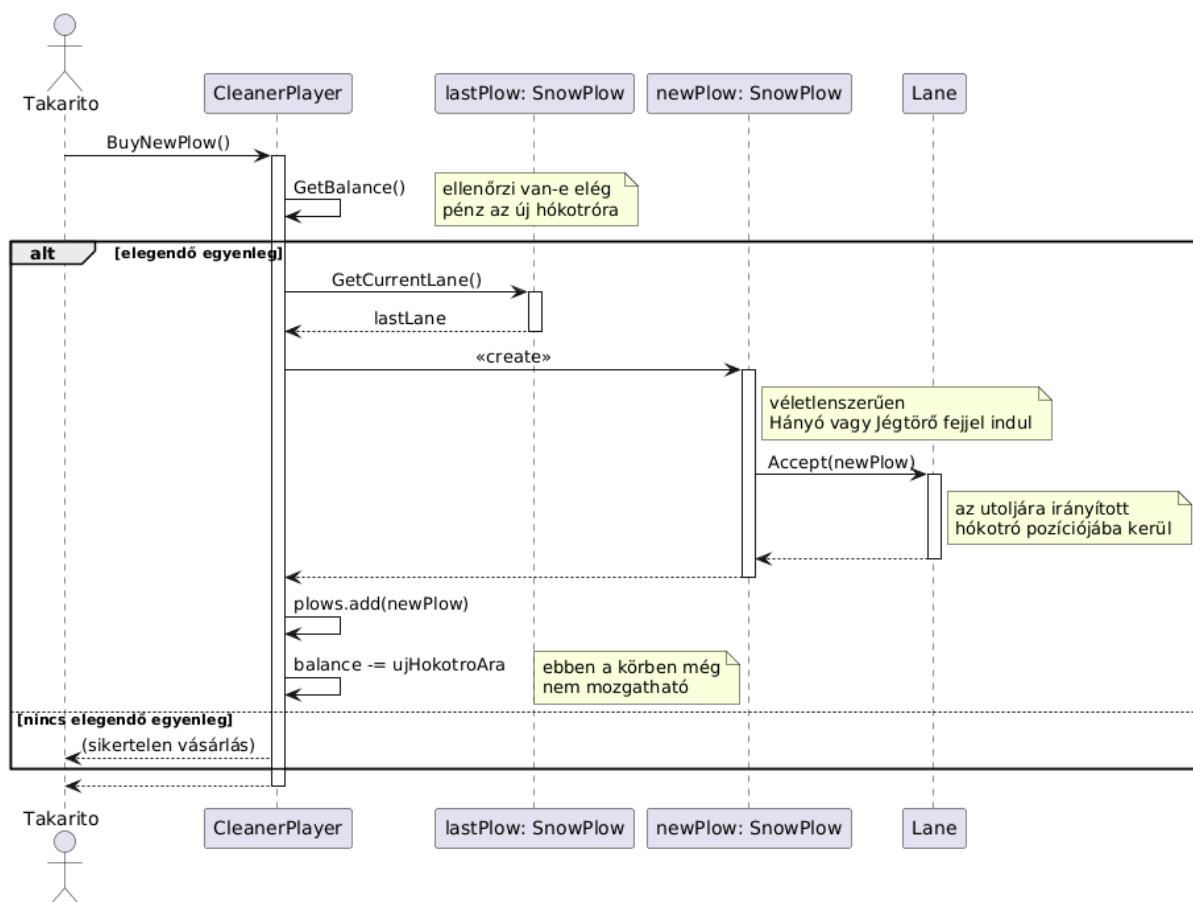
SD-15 – Fejcsere



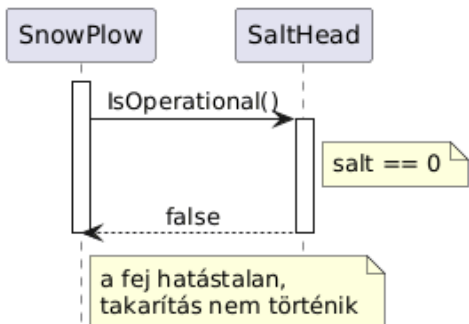
SD-15 – Hajtóanyag vásárlás



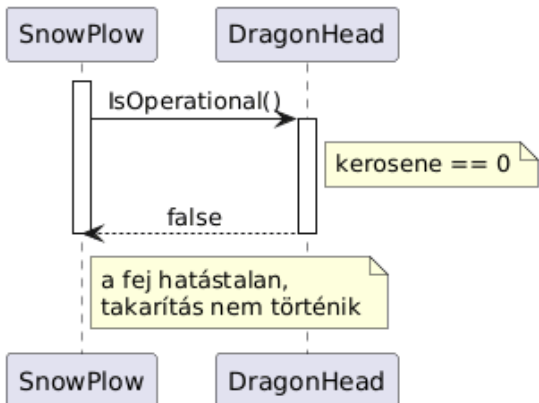
SD-16 – Új hókotró vásárlás



SD-17 – Sósóró fej kifogyása

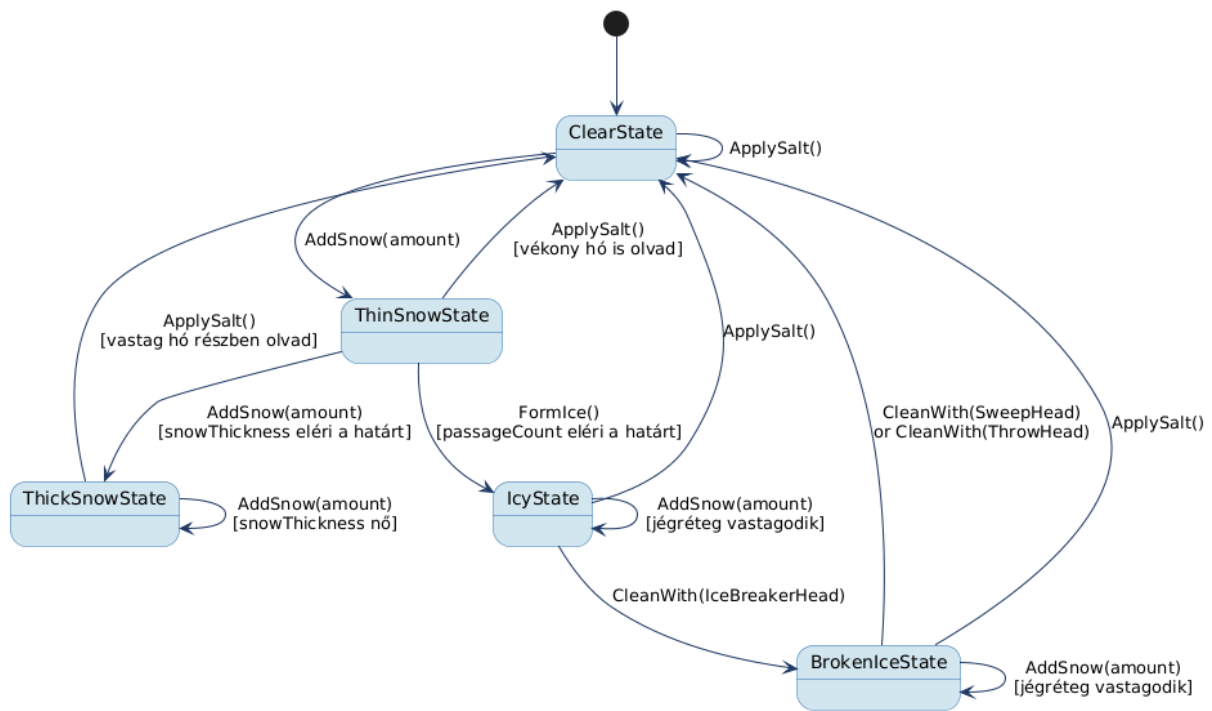


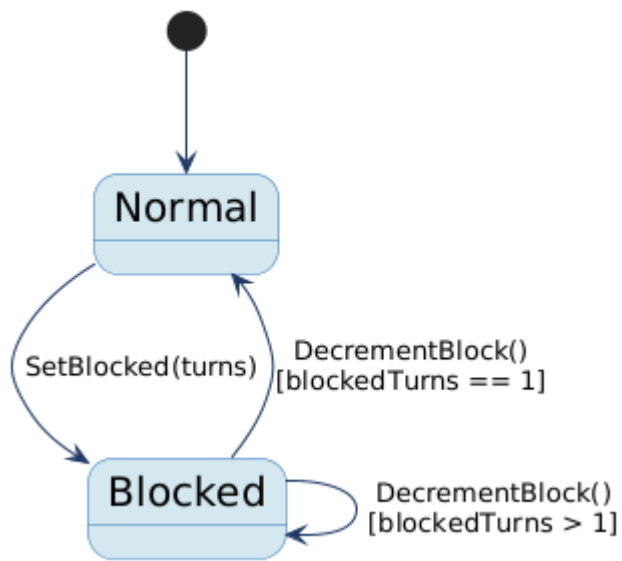
SD-18 – Sárkányfej kifogyása



3.5 State-chartok

SC-01 - Lane állapotai:



SC-02 - Vehicle blokkolt és normál állapot:

3.6 Napló

Kezdet	Időtartam	Résztevők	Leírás
2026.03.03. 18:00	2 óra	Csapat	Értekezlet. Analízis modell strukturájának megbeszélése
2026.03.04. 10:00	3 óra	Csapat	Értekezlet. Osztályok azonosítása, örökléshierarchia megtervezése. Döntés: Kiss megvalósítja az objektumkatalógust
2026.03.04. 14:00	4 óra	Csapat	Értekezlet. Szekvenciadiagramok vázlatainak elkészítése. Döntés: Stumpf és Maruzsán elkészítik a szekvencia diagramokat
2026.03.04. 19:00	2 óra	Csapat	Értekezlet. Döntés: Dekkert pontosít és elkészíti a state chartokat, Tullner a végén jóváhagyja a dokumentumot
2026.03.08. 14:00	5 óra	Kiss	Objektum katalógus megvalósítása, osztálydiagram javítása
2026.03.08. 15:30	2 óra	Stumpf	Szekvenciadiagramok készítése
2026.03.08 23:00	2 óra	Dekkert	Osztálydiagram pontosítása, dokumentum javítása, State-chartok készítése
2026.03.08 22:00	2 óra	Maruzsán	További szekvenciadiagramok készítése
2026.03.09	1 óra	Tullner Maruzsán Kiss	Dokumentum átnézése